EULER TOTIENT FUNCTION (PHI)

```
int phi(int n) {
    if (n==1) return 0;
    int ret = n;
    for (int i=2; i*i<=n; i++) {
        if (n%i==0) ret -= ret/i;
        while(n%i==0) n/= i;
    }
    if ( n != 1) ret -= ret/n;
    return ret;
}// Sieve version
vector<int> sieve_phi(int n) {
    vector<bool> is_prime(n+1,true);
    vector<int> ret(n+1);
    FOR(i,0,n) ret[i] = i;
    FOR(i,2,n) if (is_prime[i]) {
        for(int j=i; j <= n; j += i) {
            ret[j] -= ret[j]/i;
            is_prime[j] = false;}}return ret;}
void sievephi(int n) {
    phi[1] = 0;for(LL i=2; i<n; i++) phi[i] = i;
    for(LL i=2; i<=n; (i==2LL) ? i++ : i+=2LL) {
        if ( phi[i] == i ) {
            for(LL j=i; j<=n; j+=i) {
                phi[j] = phi[j] * LL((i-1LL)) / LL(i);
            }}}
    for(LL i=2; i<=n; i++) {
        for(LL j = 1, _n = n/i; j<=_n; j++)
            ans[i*j] += j * phi[i];
    }
    for(LL i=2; i<=n; i++) ans[i] += ans[i-1];
}
int phi(int n) {
    if ( n == 1 ) return 0;
    int res = n;
    for(int i=2; i<=n/i; (i==2) ? i++ : i+=2) {
        if ( n%i ) continue;
        while(n > 1 && n%i==0) n/=i;
        if ( res % i ) res = res * (i-1) / i;
        else res = res / i * (i-1);
    }
    if ( n > 1 ) res = res / n * ( n-1 );
    return res;
}
```

Extended Euclid

```
/* given a and b for equation a*x + b*y = 1, extended
Euclid will find the x and y and returns GCD(a,b) */
int ext_euclid(int a, int b, int &x, int &y) {
    if (b==0) { x=1; y=0; return a; }
    int ret = ext_euclid(b,a%b,x,y);
    int x1 = y, y1 = x-(a/b)*y;
    x=x1, y=y1;
    return ret;
}
```

LINEAR DIOPHANTINE

```
/* given a,b,and c. This function will return true if
found a solution. Need ext_euclid().. ax + by = c!,
(d,x0,y0) = ext_euclid(a,b).. x = x0 + (b/d) * n, y =
y0 - (a/d)*n; n = 0,1,2,...
bool linear_diophantine(int a,int b,int c,int &x,int
&y) {
    int d = ext_euclid(a,b,x,y);
    if ( c%d != 0) return false;
    x *= c/d; y *= c/d;
    return true;
}
```

Sum of Divisors

```
/* this function returns the sum of the divisors of n.
i.e div(3) will return sum of the number of divisors
up to 3. */
LL div(int n) {
    if (n==0) return 0;
    LL res = 0, i;
    for( i = 2; i*i <= n; i++) res += n/i;
    --i; res = (n*2) - (i*i) + (res*2);
    return res;
}
```

SUPER CATALAN & CATALAN

```
LL sc(int n) { // supercatalan
    if(n<=2) return 1;
    return (3*(2*n-3)*sc(n-1)-(n-3)*sc(n-2)) / n;
}
LL c(int n) { // catalan, 1,1,2,5,14,42,132,429,1430
    if (n<=0) return 1;
    return c(n-1)*2*(2*n+1)/(n+2);
}
```

PRIME TEST

```
#define MAX 100000000 #define HALF 50000000
#define SQRT 5000 #define SIZE MAX / 16 + 2
char SITO[SIZE];
#define NTH(n) ((n)&7)
#define ISPRIME(n) SITO[n>>4] & (1<< NTH(n>>1))
bool is_prime(int n) {
    if (n==2) return true;
    else if (!(n&1)) return false;
    else return ISPRIME(n);
}
void init() {
    FILL(SITO,0xFF);
    SITO[0] = 0xFE;
    for(int i=1; i<SQRT;i++)
        if (SITO[i>>3]&(1<<NTH(i)))
            for(int j=2*i*(i+1);j<HALF;j+=2*i+1)
                SITO[j>>3] &= ~(1 << NTH(j));
}
```

INT TO ROMAN

```
string i2r(int x) {
    const string R[] = { "M", "CM", "D", "CD", "C",
"XC", "L", "XL", "X", "IX", "V", "IV", "I" };
    const int N[] = { 1000,900,500,400,100,90,50,40,
10,9,5,4,1 };
    if (x==0) return "O"; // Not decimal 0 !
    int n = sizeof(N) / sizeof(int);
    string ret="";
    REP(i,n) while(x>=N[i]) { x-=N[i]; ret+=R[i]; }
    return ret;
}
```

nCk

```
LL res = 1;
if (n==0) { printf("%lld\n", res); continue; }
printf("%lld ", res);
for(int i=n, j=1; i>0, j<n; i--,j++) {
    res = res * i / j;
    printf("%lld ", res);
}
puts("1");
LL C(int n,int k) {
    LL numerator=1,denominator=1,toMul,toDiv,i;
    if (k>n/2) k=n-k;
    for(i=k; i; i--) {
        toMul = n-k+i;
        toDiv = i;
        GCDdiv(toMul,toDiv);
        GCDdiv(numerator,toDiv);
        GCDdiv(toMul,denominator);
        numerator *= toMul;
        denominator *= toDiv;
    }
    return numerator/denominator;
}
```

K-th SHORTEST PATH

```
const int MAXN = 100;
const int MAXK = 40;
vector<PII> g[MAXN];
priority_queue<PII> pq;
int visit[MAXN],n,k;
int solve(int s,int t) {
    FILL(visit,0);
    pq.push(MP(0,s));
    int ret=0;
    while(!pq.empty()) {
        int awal = -pq.top().first;
        int f = pq.top().second; pq.pop();
        if (visit[f]>=k) continue;
        visit[f]++;
        if (f==t) ret+=awal;
        FOREACH(it,g[f]) if (visit[it->first]<k)
        pq.push(MP(-(awal+it->second),it->first));
    } return ret;
}
int main() {
    k = GI;
    n = GI;
    REP(nedge,GI) {
        int a = GI - 1,b = GI-1,C=GI;
        g[a].PB(MP(b,c));
        g[b].PB(MP(a,c));
    }
    printf("%d\n", solve(0,n-1));
    return 0;
}/* k = shortest path ke berapa.
n = jumlah vertex.
solve(i,j); jika ingin shortest path dari i ke j.
*/
```

Articulation Point & Bridge

```
int dfn[101], low[101];
char input[1001], *ptr;
int num, at;
void dfs( vector< vector< int > > &v, int node, int
par ){
    vector< int > :: iterator it;
    dfn[node] = low[node] = num;
    num++;
    int numChild = 0;
    bool art = false;
    for( it = v[node].begin() ; it != v[node].end();
it++ ){
        if( dfn[*it] == -1 ){
            dfs(v,*it,node);
            low[node] = min( low[node], low[*it] );
            numChild++;
            if( dfn[node] == 1 ){
                if( numChild >= 2 ) art = true;
            } else if( low[*it] >= dfn[node] ) art =
true;
        } else if( *it != par ) {
            low[node] = min( low[node], dfn[*it] );
        }
    }
    if( art ) at++;
    // kalo mau tau vertexnya, ganti jadi
    // if ( art ) ans.push_back(node);
}
int main(){
    int n, f, t, i;
    while(scanf("%d\n", &n ) == 1 ){
        if( n == 0 ) break;
        num = 1;
        for( i = 0 ; i < n + 1 ; i++ ) {
            dfn[i] = -1, low[i] = -1;
        }
        vector< vector< int > > v( n + 1 );
        while(gets( input) ){
            if( strcmp( input, "0" ) == 0 ) break;
            ptr = strtok( input, " " );
            f = atoi( ptr );
            while( 1 ){
                ptr = strtok( NULL, " " );
                if( ptr == NULL ) break;
                t = atoi( ptr );
                v[f].push_back(t);
                v[t].push_back(f);
            }
        }
        at = 0;
        dfs(v,1,1);
        printf("%d\n", at );
    }
    return 0;
}
// Art Point
vector< int > v[1002];
int nvis, inStack[1002], low[1002], idx[1002],
ncount[1002];

void tarjan( int nv ) {
    idx[nv] = low[nv] = nvis++;
    inStack[ nv ] = 1;
    for( int i=0; i<v[nv].size(); i++ ) {
        int u = v[nv][i];
        if ( !inStack[u] ) {
            tarjan( u );
            if ( idx[nv] <= low[u] ) ncount[nv]++;
            low[nv] = min( low[nv], low[u] );
        } else {
            low[nv] = min( low[nv], idx[u] );
        }
    }
}

int main() {
    int a,b, TC = 1;
    set< int > st;
    while( scanf("%d", &a) != EOF ) {
        if ( !a ) break;
        st.clear();
        scanf("%d", &b);
        st.insert( a-1 );
        st.insert( b-1 );
        v[a-1].push_back( b-1 );
        v[b-1].push_back( a-1 );
        while( scanf("%d", &a) != EOF ) {
            if ( !a ) break;
            scanf("%d", &b);
            st.insert( a-1 );
            st.insert( b-1 );
            v[a-1].push_back( b-1 );
            v[b-1].push_back( a-1 );
        }
        memset(ncount, 0, sizeof(ncount) );
        memset(inStack, 0, sizeof(inStack) );
        nvis = 1;
        tarjan( 0 );
        ncount[0]--;
        printf("  SPF node %d leaves %d subnets\n", i
+ 1, ncount[i] + 1 );
    }
    return 0;
}
// Bridge
int cost[20][20], N, M;
int nvis, inStack[20], low[20], idx[20], p[20];
vector< pair<int, pair<int,int> > > v;
vector< pair<int,int> > bedge;
void tarjan( int nv ) {
```

```
    idx[nv] = low[nv] = nvis++;
    inStack[ nv ] = 1;
    for( int i=1; i<=N; i++ ) {
        if ( cost[nv][i] <= 0 ) continue;
        int u = i;
        if ( !inStack[u] ) {
            p[u] = nv;
            tarjan( u );
            if ( idx[nv] < low[u] )
{bedge.push_back( make_pair( nv, u ) );}
            low[nv] = min( low[nv], low[u] );
        } else if ( u != p[nv] ) {
            low[nv] = min( low[nv], idx[u] );
        }
    }
}
```

DIRECTED MST

```
int con[MAXN][MAXN], in[MAXN];
int vis[MAXN], active[MAXN], n;
// vis state: 0:unvisited, 1:visited, 2:cycle,
3:active
int cycle(int idx, int &res){
    if (vis[idx]==0) {
        vis[idx] = 3;
        vis[idx]=(in[idx] !=-1)?cycle(in[idx],res): 1;
    } else if (vis[idx]==3){
        set<int> pseudoNodes;
        int minCycle = INF;
        for (int j=idx; ; j=in[j]){
            pseudoNodes.insert(j);
            minCycle <?= con[in[j]][j];
            if (in[j]==idx) break;
        }
        int minCost = INF, bestParent;
        FOREACH(it,pseudoNodes){
            int j = *it; assert(active[j]);
            REP(i,n) if (active[i] && con[i][j]!=-1 &&
            !pseudoNodes.count(i)) {
             if (minCost > con[i][j] - con[in[j]][j]){
                minCost = con[i][j] - con[in[j]][j];
                bestParent = i;
             }
              con[i][j] -= con[in[j]][j] - minCycle;
            }
        }
        res += minCost;
        in[idx] = bestParent;
        FOREACH(it,pseudoNodes){
            int j = *it;
            REP(i,n) if (active[i] && i!=idx &&
                con[i][j]!=-1){
                    if (con[i][idx]==-1 ||
                        con[i][idx]>con[i][j])
                            con[i][idx] = con[i][j];
                }
            REP(i,n) if (active[i] && i!=idx &&
                    con[j][i]!=-1){
                        if (in[i]==j) in[i] = idx;
                            if (con[idx][i]==-1 ||
                            con[idx][i]>con[j][i])
                            con[idx][i] = con[j][i];
            }
            if (j!=idx) active[j] = false;
        }
        vis[idx] = 2;
    }
    return vis[idx];
}
int dmst(int root){
    int res = 0;
    in[root] = -1;
    REP(i,n) con[i][root] = -1;
    REP(i,n) if (i!=root) {
        int minCost = INF;
        REP(j,n) if (con[j][i]!=-1) {
            if (con[j][i] < minCost) {
                minCost = con[j][i];
                in[i] = j;
            }
        }
        res += minCost;
    }
    memset(active,1,sizeof(active));
    while (true){
        memset(vis,0,sizeof(vis));
        bool hasCycle = cycle(root,res) == 2;
        REP(i,n) if (active[i] && !vis[i])
            hasCycle |= cycle(i,res) == 2;
        if (!hasCycle) return res;
    }
}
int dfs(int idx){
    if (vis[idx]) return 0; else vis[idx] = true;
    int ret = 1;
    REP(i,n) if (con[idx][i]!=-1) ret += dfs(i);
    return ret;
}
bool connected(int root){
    memset(vis,0,sizeof(vis));
    return dfs(root) == n;
}
int main(){
    int nTC,m,u,v,w;
    scanf("%d",&nTC);
    for (int TC=1; TC<=nTC; TC++){
        printf("Case #%d: ",TC);
        scanf("%d %d",&n,&m);
        memset(con,-1,sizeof(con));
        REP(i,m){
            scanf("%d %d %d",&u,&v,&w);
            con[u][v] = w;
        }
        if (!connected(0)) puts("Possums!");
        else printf("%d\n", dmst(0));
    }
}
```

CLASS POLYGON

```
#define VP vector<Point>
#define EPS 1e-9
#define sqr(x) ((x)*(x))
#define dist2(p1,p2) (sqr(p1.x-p2.x) + sqr(p1.y-p2.y))
#define det(p1,p2,p3) (p1.x * p2.y + p2.x * p3.y
+p3.x*p1.y-p1.y * p2.x-p2.y * p3.x-p3.y * p1.x)
class Point {
    public :
    double x,y;
    Point(){}
    Point(double _x, double _y):x(_x),y(_y){}

    double dist(Point that){
        double dx = x - that.x;
        double dy = y - that.y;
        return sqrt(dx*dx + dy*dy);
    }

    bool operator<(Point const &that) const{
        double d;
        if (fabs(d=(x-that.x))>EPS) return d<0;
        if (fabs(d=(y-that.y))>EPS) return d<0;
        return false;
    }
};
class PivotCompare {
    public :
```

```
    Point pv;
    PivotCompare(Point _pv):pv(_pv){}
    bool operator()(Point p1, Point p2){
        if (det(pv,p1,p2)==0) return dist2(pv,p1) <
dist2(pv,p2);
        return atan2(p1.y-pv.y, p1.x-pv.x) <
atan2(p2.y-pv.y, p2.xpv.x);
    }
};
VP convex_hull(VP p) {
    if (p.size()<3) return VP();
    FOR(i,1,p.size()-1)
    if (p[i].y < p[0].y || p[i].y == p[0].y &&
    p[i].x < p[0].x) swap(p[0], p[i]);
    Point pv = p[0];
    sort(p.begin()+1, p.end(), PivotCompare(pv));
    VP cp(p.begin(),p.begin()+2);
    FOR(i,2,p.size()-1){
        while (cp.size() > 1 && det(cp[cp.size()-
2],cp.back(),p[i]) <= 0)
            cp.pop_back();
        cp.push_back(p[i]);
    }
    return cp;
}
class Line {
    public :
    Point p1, p2;
    Line(){}
    Line(Point _p1, Point _p2):p1(_p1),p2(_p2){}
    bool intersect(double x1, double y1, double x2,
double y2, double x3, double y3, double x4, double
y4,double &x, double &y) {
       double nom = (x2-x1)*(y1-y3) + (y2-y1)*(x3-x1);
        double den = (x2-x1)*(y4-y3)-(x4-x3)*(y2-y1);
        if (den==0) return false;
        double b = nom/den;
        x = x3 + (x4-x3) * b;
        y = y3 + (y4-y3) * b;
        double a = (x2-x1 == 0) ? ((y-y1) / (y2-y1)) :
((x-x1) / (x2-x1));
        return 0<=a && a<=1 && 0<=b && b<=1;
    }
    bool intersect(Line &that, Point &p) {
        return intersect(p1.x, p1.y, p2.x, p2.y,
that.p1.x, that.p1.y, that.p2.x, that.p2.y, p.x, p.y);
    }
};
class Polygon {
    public :
    VP P;
    Polygon(){}
    Polygon(VP _P):P(_P){}
    void addPoint(Point p){
        P.push_back(p);
    }

    Polygon intersect(Polygon &that) {
        double qarea = that.area(), parea = area();
        VP &Q = that.P;
        set<Point> p;
        Point pt;
        REP(i,P.size()){
            int ni = (i+1)%P.size();
            Point p1(P[i].x, P[i].y),
p2(P[ni].x,P[ni].y);
            Line L1(p1, p2);
            REP(j,Q.size()){
                int nj = (j+1)%Q.size();
                Point p3(Q[j].x, Q[j].y),
p4(Q[nj].x,Q[nj].y);
                Line L2(p3, p4);
                if (L1.intersect(L2,pt)) p.insert(pt);
            }
        }
        REP(i,P.size()){
            double a = 0; REP(j,Q.size()) a +=
fabs(det(Q[j],
            Q[(j+1)%Q.size()], P[i])/2);
            if (fabs(a-qarea) < EPS) p.insert(P[i]);
        }
        REP(i,Q.size()){
            double a = 0; REP(j,P.size()) a +=
fabs(det(P[j],
            P[(j+1)%P.size()], Q[i])/2);
            if (fabs(a-parea) < EPS) p.insert(Q[i]);
        }
        return
Polygon(convex_hull(VP(p.begin(),p.end())));
    }
    double area(){
        double ret = 0;
        REP(i, P.size()){
            int ni = (i+1)%P.size();
            ret += P[i].x * P[ni].y - P[i].y * P[ni].x;
        }
        return fabs(ret/2);
    }
};

int main(){
    for (int nA, nB, x, y; scanf("%d",&nA)!=EOF &&
nA; ) {
        Polygon P, Q;
        REP(i,nA) {
            scanf("%d %d",&x,&y);
            P.addPoint(Point(x,y));
        }
        scanf("%d",&nB);
        REP(i,nB) {
            scanf("%d %d",&x,&y);
            Q.addPoint(Point(x,y));
        }
        Polygon I = P.intersect(Q);
        printf("%8.2lf\n", P.area() + Q.area() - 2 *
I.area());
    }
    puts("");
    return 0;
}
```

HUNGARIAN ALGORITHM

```
int cost[64][64], lvisit[64], rvisit[64];
int lvalue[64], rvalue[64], rmatch[64], n;
inline bool findmatch(int s) {
    lvisit[s] = true;
    REP(i,n) {
        if(rvisit[i]||lvalue[s]+rvalue[i]!=cost[s][i])
continue;
        rvisit[i] = true;
        if(rmatch[i]==-1 || findmatch(rmatch[i])) {
            rmatch[i] = s;
            return true;
        }
    }
    return false;
}
inline void normalize() {
    int down = INF;
    REP(i,n) {
        if(lvisit[i]) {
            REP(j,n) {
                if(!rvisit[j]) down =
min(down,lvalue[i]+rvalue[j]-cost[i][j]);
            }
```

```
        }
    }
    REP(i,n) {
        if(lvisit[i]) lvalue[i] -= down;
        if(rvisit[i]) rvalue[i] += down;
    }
}
inline int calc() {
    REP(i,n) {
        lvalue[i] = *max_element(cost[i],cost[i]+n);
        rvalue[i] = 0;
    }
    REP(i,n) rmatch[i] = -1;
    REP(i,n) {
        FILL(lvisit,0);
        FILL(rvisit,0);
        if(findmatch(i)) continue;
        i--;
        normalize();
    }
    int ret = 0;
    REP(i,n) ret += cost[rmatch[i]][i];
    return ret;
}
```

LIS (LONGEST INCREASING SUBSEQUENCE)

```
vector<int> find_lis(vector<int> &a) {
    vector<int> b, p(a.size());
    int u, v;
    if (a.size() < 1) return b;
    b.push_back(0);
    for (int i = 1; i < (int)a.size(); i++) {
        if (a[b.back()] < a[i]) {
            p[i] = b.back();
            b.push_back(i);
            continue;
        }
        for (u = 0, v = b.size()-1; u < v;) {
            int c = (u + v) / 2;
            if (a[b[c]] < a[i]) u=c+1; else v=c;
        }
        if (a[i] < a[b[u]]) {
            if (u > 0) p[i] = b[u-1];
            b[u] = i;
        }
    }
    for (u = b.size(), v = b.back(); u--; v = p[v])
b[u] = v;
    return b;
}
```

TOPOLOGICAL SORT

```
void topoSort(int v) {vis[v] = 1;
for(int i=0; i<ad[v].size(); i++) {if
( !vis[ ad[v][i] ] ) {topoSort(ad[v][i]);}}
topoSorted.push_back(v);}
int main() {
    topoSorted.clear();
    REP(i,N) if ( !vis[i] ) topoSort(i);
    reverse(topoSorted.begin(), topoSorted.end());
}
```

LCIS (LONGEST COMMON INCREASING SUBSEQUENCE)

```
typedef long long LL;
const int INF = 1000000000;
const LL INFLL = (LL)INF*(LL)INF;
inline int getint() { int x; scanf("%d",&x); return
x; }
const int MAXN = 512;
int A[MAXN]; int na;
int B[MAXN]; int nb;
int prev[MAXN][MAXN];
LL best[MAXN][MAXN];
int pos[MAXN][MAXN];
inline void printPath(int path) {
    if ( prev[path>>10][path&1023] == -1 )
        printf("%d",A[path>>10]);
    else {
        printPath(prev[path>>10][path&1023]);
        printf(" %d",A[path>>10]);
    }
}
int main() {
    REP(i,na=getint()) A[i]=getint();
    REP(i,nb=getint()) B[i]=getint();
    REP(i,nb) REP(j,nb) best[i][j] = INFLL;
    REP(i,na) REP(j,nb) prev[i][j] = -1;
    REP(i,na) {
        int x = -1, p = 0;
        REP(j,nb) {
            if ( A[i] == B[j] ) {
                int y = p;
                while(best[j][y]<A[i]) y++;
                best[j][y] = A[i];
                pos[j][y] = (i<<10)+j;
                if ( y > 0 ) {
                    prev[i][j]=pos[j][y-1];
                }
                x = p = y;
            } else {
                if ( x != -1 && best[j-1][x] <
best[j][x] ) {
                    best[j][x] = best[j-1][x];
                    pos[j][x] = pos[j-1][x];
                } else x=-1;
            }
        }
    }
    int len=-1;
    FORD(i,nb-1,0) if(best[nb-1][i]!=INFLL) { len = i;
break; }
    printf("%d\n",len+1);
    if ( len >= 0 ) printPath(pos[nb-1][len]);
    puts("");
    return 0;
}
```

MISOF TREE

```
class KthTree {
private:
int **tree,nlv;
int lv( int n ) {int ret = 0;while( (1 << ret) <= n )
ret++;return ret;}
public:
KthTree( int n ) {this->nlv = this->lv( n ) + 1;
tree=(int**)malloc(sizeof(int*)*(this->nlv));
for(int i=0;i<this->nlv;i++) {tree[i] =
(int*)malloc(sizeof(int)*(1<<(this->nlv-1)));
memset(tree[i],0,sizeof(int)*(1<<(this->nlv-1)));}}
~KthTree(){for(int i=0;i<this->nlv;i++)
free(tree[i]);free(tree);}
void insert(int n){for(int i=0;i<this-
>nlv;i++){tree[i][n]++; n /= 2;}}
void erase( int n ) {for(int i=0;i<this-
>nlv;i++){tree[i][n]--;n /= 2;}}
int KthElement(int k) {int a=0,b=(this->nlv)-
1;while(b--){a*=2;if ( tree[b][a] < k ) k -=
tree[b][a++];}return a;}
};
```

TRIES

```
typedef struct{
 char kar;
 bool isWord;
 int parent;
 map<char,int> child;
}node;
```

```
char s[100];
node tree[2000001];
int idx, len, nn, ma;
void insert(int nodeIdx, int charIdx){
 if (tree[nodeIdx].child.count(s[charIdx])==0){
  tree[++idx].kar = s[charIdx];
  tree[idx].parent = nodeIdx;
  tree[idx].child.clear();
  tree[nodeIdx].child[s[charIdx]] = idx;
 }
 if (charIdx==len-1)
tree[tree[nodeIdx].child[s[charIdx]]].isWord = 1;
 else
 insert(tree[nodeIdx].child[s[charIdx]], charIdx+1);
}

void traverse(int x,int depth){
 map<char,int>::iterator i;
 depth+=tree[x].isWord;
 if (ma < depth)
 {
  ma = depth;
  nn = x;
 }

for(i=tree[x].child.begin();i!=tree[x].child.end();i++)
  traverse(i->second,depth);
}

int main()
{
 idx = 0;
 tree[0].kar=0;
 ma=-1;
 while (scanf("%s",s))
 {
  if (strcmp(s,".")==0) break;
  len = strlen(s);
  insert(0,0);
 }
 nn=0;
 traverse(0,0);
 int  res[100];
 char ss[100];
 int dd=0;
 while (nn)
 {
  ss[dd]= tree[nn].kar;
  res[dd] = tree[nn].isWord;
  dd++;
  nn = tree[nn].parent;
 }
 reverse(ss,ss+dd);
 reverse(res,res+dd);
 ss[dd] = 0;

 for(int i=0;i<dd;i++){
  if (res[i]){
   char temp = ss[i+1];
   ss[i+1] = 0;
   printf("%s\n",ss);
   ss[i+1] = temp;
  }
 }
 return 0;
}
// input for above
i
if
in
input
int
integer
output
// the output is
i
in
int
integer
// Another trie
#define MA 5000010
typedef struct Tnode{
 int counter;
 char p;
 int child[3];
}node;
char s[210];
node trie[MA];
int idx,len;
int res;

void insert(int trid,int id){
 trie[trid].counter++;
 if (id == len) return;
 if (trie[trid].child[s[id]-'0']==-1){
  idx++;
  trie[idx].p = s[id];
  trie[trid].child[s[id]-'0'] = idx;
 }
 insert(trie[trid].child[s[id]-'0'], id+1);
}

void calc(int idx,int length){
 res = max(res, trie[idx].counter * length);
 if (trie[idx].child[0]!=-1)
calc(trie[idx].child[0],length+1);
 if (trie[idx].child[1]!=-1)
calc(trie[idx].child[1],length+1);
}

int main(){
 int  t,n;
 scanf("%d",&t);
 while (t--){
  idx = 0;
  scanf("%d",&n);
  for(int i=0;i<MA;i++)
   trie[i].counter = 0,trie[i].child[0] =
trie[i].child[1] =-1;

  while (n--){
   scanf("%s",s);
   len = strlen(s);
   insert(0,0);
  }
  res = 0;
  calc(0,0);
  printf("%d\n",res);
 }
}
```

EDIT DISTANCE

```
int dp[MAXN][MAXN];
char s1[MAXN],s2[MAXN];
int main() {
    int len1,len2;
    while(scanf("%d", &len1)!=EOF) {
        if (len1>0) scanf("%s", s1);
        scanf("%d", &len2);
        if (len2>0) scanf("%s", s2);
        bool one = (len1==0) ? 1 : 0, two =
(len2==0)?1 : 0;
        FILL(dp,0);
        dp[0][0] = (s1[0]==s2[0]) ? 0 : 1;
```

```
      FOR(i,1,len1-1) dp[i][0] = (s1[i]==s2[0]) ? i :
MIN(i+1,dp[i-1][0]+1);
      FOR(i,1,len2-1) dp[0][i] = (s1[0]==s2[i]) ? i :
MIN(i+1,dp[0][i-1]+1);
       FOR(i,1,len1-1) {
           FOR(j,1,len2-1) {
               dp[i][j] = INF;
               if (s1[i]==s2[j]) dp[i][j] <?= dp[i-
1][j-1];
               dp[i][j] = MIN(dp[i][j],MIN(dp[i-1][j]
+ 1, MIN(dp[i][j-1] + 1,dp[i-1][j-1] + 1)));
           }
       }
       printf("%d\n", (one^two) ? (max(len1,len2)) :
dp[len1-1][len2-1]);
    }
}
```

DIGIT COUNTING FACTORIAL

```
#include<stdio.h>
#include<math.h>
double memo[11000];
void init() {
    memo[1] = log10(1);
    for(int i = 2; i<=10000; i++) {
       memo[i] = (double)memo[i-1] + (double)log10(i);
    }
}
int main() {
    init();
    int n;
    scanf("%d", &n);
    double jmldigit = 0.0;
    int jmlnol = 0;
    for(int i=0; i<n; i++) {
        int x;
        scanf("%d", &x);
        jmldigit += memo[x];
    }
    printf("%d\n", ((int)jmldigit)+1);
} // How many digits exist in n! :D
```

KTH-PERMUTATION

```
string kthpermute(LL k, string s) {
    int len = s.length();
    char ss[len+2];
    sprintf(ss, "%s", s.c_str());
    LL fac = 1LL;
    FOR(i,2,len-1) fac = LL(i) * LL(fac);
    REP(i, len-1) {
        int tidx = ( k / fac ) % ( len - i );
        char ch = ss[ i + tidx ];
        for(int j = i + tidx; j > i; j--) ss[j] =
ss[j-1];
        ss[i] = ch;
        fac = fac / LL( (len - i - 1) );
    }
    return (s = ss);
}
int main() {
    string s = "abcdefghijklmnopqrst";
    printf("%s\n", kthpermute(123456789987654321LL,
s).c_str() );
    return 0;
}
```

DP Buka String & Lipat String

```
struct tipetipe {int min, nkurung;};
int n;
string buka( int &i, string s ) {
    int len=s.length(), angka=0;
    string tmp;
    while ( i<len && s[i] != ')' ) {
        if ( s[i]>='0' && s[i]<='9' ) {
            angka = angka*10 + (s[i]-'0');
        }
        else if ( s[i] == '(' ) {
            i++;
            string tmp2 = buka( i, s );
            REP(i,angka) tmp += tmp2;
            angka = 0;
        }
        else {
            tmp = tmp+s[i];
        }
        i++;
    }
    return tmp;
}
tipetipe memo[305][305];
int path[305][305][4];
string s2;
tipetipe hohoho( int l, int r ) {
    //printf( "%d %d\n", l, r );
    //fflush(stdin);
    //getchar();
    if ( memo[r][l].min != -1 ) return memo[r][l];
    tipetipe ret;
    ret.min = r-l+1;
    ret.nkurung = 0;
    path[l][r][0] = 0;

    FOR( i,1,(r-l+1)/2 ) if ( (r-l+1)%i == 0 ) {
        string t1 = s2.substr( 0+l, i );
        string t2 = s2.substr( i+l, i );
        int ni=1, j=i+l;
        while ( t1 == t2 && j<=r ) {
            ni++; j+=i;
            t2 = s2.substr( j, i );
        }
        if ( ni == (r-l+1)/i ) {
            int dig=0;
            int nii=ni;
            while (ni>0) dig++, ni/=10;
            tipetipe tmp;
            tmp = hohoho(l,l+i-1);
            tmp.min += dig+2;
            tmp.nkurung += 2;
            if ( tmp.min < ret.min ) {
                ret = tmp;
                path[l][r][0] = 1;
                path[l][r][1] = nii;
                path[l][r][2] = i;
            }
            else if ( tmp.min == ret.min && tmp.nkurung
< ret.nkurung ) {
                ret = tmp;
                path[l][r][0] = 1;
                path[l][r][1] = nii;
                path[l][r][2] = i;
            }
        }
    }
    FOR( i,l,r-1 ) {
        tipetipe tmp, left, right;
        left = hohoho(l,i); right = hohoho(i+1,r);
        tmp.min = left.min + right.min;
        tmp.nkurung = left.nkurung + right.nkurung;
        if ( tmp.min < ret.min ) {
            ret = tmp;
            path[l][r][0] = 2;
            path[l][r][1] = i;
        }
        else if ( tmp.min == ret.min && tmp.nkurung <
ret.nkurung ) {
            ret = tmp;
```

```
            path[l][r][0] = 2;
            path[l][r][1] = i;
        }
    }
    //printf( "l->%d   r->%d   res->%d  path->%d\n",
l,r,ret,path[l][r][0] );
    return memo[r][l] = ret;
}
string search_path( int l, int r ) {
    string ret;
    if ( path[l][r][0] == 0 ) {
        FOR( i,l,r ) ret += s2[i];
    }
    if ( path[l][r][0] == 1 ) {
        char tmpx[10];
        sprintf( tmpx, "%d", path[l][r][1] );
        ret += tmpx;
        ret += "(" + search_path( l, l+path[l][r][2]-
1 ) + ")";
    }
    if ( path[l][r][0] == 2 ) {
        ret += search_path( l, path[l][r][1] );
        ret += search_path( path[l][r][1]+1, r );
    }
    return ret;
}

int main() {
    cin >> n;
    REP( i,n ) {
        string s1;
        cin >> s1;
        int z = 0;
        s2 = buka(z,s1);
        memset(memo,-1,sizeof(memo));
        hohoho( 0, s2.length()-1 );
        cout << s2 << endl;
        cout << search_path( 0, s2.length()-1 ) <<
endl;
    }
    return 0;
}
// input
2(haue)yueuyeuue3(xxzz)zzxx
hauehaueyueuyeuuexxzzxxzzxxzzzzxx
2(haue)yueuyeuue3(xxzz)zzxx
```

MATRIX CHAIN MULTIPLICATION

```
#define MAX 100
#define inf 2000000000
int p[MAX+1];
int m[MAX+1][MAX+1],s[MAX+1][MAX+1];
void MCM(int n) {
    int l, j, i, k, q;
    for(i=0;i<=n;i++)m[i][i]=0;
    for(l=2;l<=n;l++) {
        for(i=1;i<=n-l+1;i++) {
            j=i+l-1;
            m[i][j]=inf;
            for(k=i;k<=j-1;k++) {
                q=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
                if( q < m[ i ][ j ] ) {
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }
}
void print_MCM(int i,int j) {
    if(i == j) printf("A%d", i);
    else {
        printf("(");
        print_MCM(i,s[i][j]);
        printf(" x ");
        print_MCM(s[i][j]+1,j);
        printf(")");
    }
}
int main(){
    int n, i, j, cs = 1;
    while( scanf("%d", &n ) == 1 ) {
        if( n == 0 ) break;
       for(i=0;i<n;i++)scanf("%d %d", &p[i], &p[i+1]);
        printf("Case %d: ", cs++ );
        MCM( n );
        print_MCM( 1, n  );
        puts("");
    }
    return 0;
}
```

HUFFMAN CODES

```
#include <fstream>
using namespace std;
typedef long long ll;
int main() {
    priority_queue<ll, vector<ll>, greater<ll> > q;
    ifstream in("plank.in");
    ofstream out("plank.out");
    int N;
    in >> N;
    for (int i = 0; i < N; i++) {
        ll x;
        in >> x;
        q.push(x);
    }
    ll cost = 0;
    for (;;) {
        ll a = q.top();
        q.pop();
        if (q.empty()) break;
        ll b = q.top();
        q.pop();
        cost += a + b;
        q.push(a + b);
    }
    out << cost << "\n";
    return 0;
}
```

RUMUS SEGITIGA

```
Tan(A+B)     = (Tan A + Tan B) / (1 - Tan A Tan B)
Tan(A-B)     = (Tan A - Tan B)/(1 + Tan A Tan B )
Tan(2A)     = (2 Tan A)/(1-Tan² A)
Sin(3A)     = -4 Sin³ A + 3 Sin A
Cos(3A)     = 4 Cos³ A - 3 Cos A
Sin(A/2)     = ((1 - Cos A)/2)^0.5
Cos(A/2)     = ((1 + Cos A)/2)^0.5
Tan(A/2)     = ((1 - Cos A)/(1 + Cos A))^0.5
Tan A + Tan B + Tan C     = Tan A Tan B Tan C
A + B + C = 180
Sec² A - Tan² A     = 1
Csc² A - Cot² A     = 1
2 Sin A Cos B     = Sin(A+B) + Sin(A-B)
2 Cos A Sin B     = Sin(A+B) - Sin(A-B)
2 Cos A Cos B     = Cos(A+B) + Cos(A-B)
-2 Sin A Sin B     = Cos(A+B) - Cos(A-B)
Sin A + Sin B     = 2 Sin((A+B)/2) Cos((A-B)/2)
Sin A - Sin B     = 2 Cos((A+B)/2) Sin((A-B)/2)
Cos A + Cos B     = 2 Cos((A+B)/2) Cos((A-B)/2)
Cos A - Cos B     = -2 Sin((A+B)/2) Sin((A-B)/2)
```

Rumus Segitiga

```
a = panjang sisi seberang titik A
b = panjang sisi seberang titik B
c = panjang sisi seberang titik C
```

Aturan Sinus
a/Sin A = b/Sin B = c/Sin C
Aturan Cosinus
a² = b² + c² − 2 bc Cos A
b² = a² + c² − 2 ac Cos B
c² = a² + b² − 2 ab Cos C
Luas Segitiga Sembarang
L = (ab Sin C)/2 = (bc Sin A)/2 = (ac Sin B)/2
Luas Segi n Sembarang
L = (n r r Sin(360/n))/2
Luas Segitiga dengan 3 sisi
L = (s(s-a)(s-b)(s-c))^0.5
S = (a+b+c)/2
Lingkaran Dalam Segitiga
Rd = Luas Segitiga / s
Lingkaran Luar Segitiga
Rl = abc / 4 Luas Segitiga
Lingkaran Singung Segitiga
Sisi a: (s(s-b)(s-c)/(s-a))^0.5
Sisi b: (s(s-a)(s-c)/(s-b))^0.5
Sisi c: (s(s-a)(s-b)/(s-c))^0.5
Titik ke Garis
U = ((x3-x1)(x2-x1)+(y3-y1)(y2-y1))/((x2-x1)²+(y2-y1)²)
X = x1 + U(x2-x1)
Y = y1 + U(y2-y1)

KMP ALGORITHM

```
char T[MAX], P[MAX], pre[MAX];
int KMP() {
    int i, j, k;
    j = pre[0] = -1;
    for ( i=1; P[i]; i++ ) {
        while( j!=-1&&P[j+1]!=P[i]) j=pre[j];
        if ( P[j+1] == P[i] )j++;
        pre[i] = j;
    }
    k=-1;
    for(j=0,i--;T[j];j++) {
        while(k!=-1&&P[k+1]!=T[j]) k = pre[k];
        if ( P[k+1] == T[j] ) k++;
        if ( k == i ) return 1;
    }
    return 0;
}
```

FENWICK TREE

```
vector<int> create(int n) { return vector<int>(n, 0); }
int query(const vector<int> &tree, int a, int b) {
    if (a == 0) {
        int sum = 0;
        for (; b >= 0; b = (b & (b + 1)) - 1){sum +=
tree[b];}return sum;
    } else {return query(tree, 0, b) - query(tree, 0,
a-1);}}
void increase(vector<int> &tree, int k, int inc) {
    for (; k <(int)tree.size();k|=k+1)tree[k] += inc;
}
int main() {
    vector<int> tree = create(10000);
    increase(tree,2,3);
    int sum = query(2,3);
    printf("%d\n", sum);
}
// 2D Version !!
int tree[1030][1030];
int mat[1030][1030];
int n;
char cmd[5];
void update( int x, int y, int val ) {
    int y1;
    while ( x <= n ) {
        y1 = y;
        while ( y1 <= n ) {
            tree[x][y1] += val;
            y1 += (y1&-y1);
        }
        x += (x&-x);
    }
}
int read( int x, int y ) {
    int y1;
    int sum = 0;
    while ( x > 0 ) {
        y1 = y;
        while ( y1 > 0 ) {
            sum += tree[x][y1];
            y1 -= (y1&-y1);
        }
        x -= (x&-x);
    }
    return sum;
}
int main() {
    scanf( "%d", &n );
    scanf( "%s", cmd );
    while ( strcmp(cmd,"END") != 0 ) {
        if ( strcmp(cmd,"SET") == 0 ) {
            int x,y,v;
            scanf( "%d %d %d", &x, &y, &v );
            x++; y++;
            update( x, y, -mat[x][y] );
            update( x, y, v );
            mat[x][y] = v;
        }
        if ( strcmp(cmd,"SUM") == 0 ) {
            int x1,y1,x2,y2;
            scanf( "%d %d %d %d", &x1, &y1, &x2, &y2 );
            x1++; y1++; x2++; y2++;
            int res = read( x2,y2 );
            if ( x1-1>0 ) res -= read( x1-1, y2 );
            if ( y1-1>0 ) res -= read( x2, y1-1 );
            if ( x1-1>0 && y1-1>0 ) res += read( x1-1,
y1-1 );
            printf( "%d\n", res );
        }
        scanf( "%s", cmd );
    }
}
```

STABLE MARRIAGE

```
int m, n;
int L[max_man][max_woman],
R[max_woman][max_man];
int L2R[max_man], R2L[max_woman];
int p[max_man];
void stableMarriage() {
    static int p[128];
    memset(R2L, -1, sizeof(R2L));
    memset(p, 0, sizeof(p));
    for( int i = 0; i < m; i++ ) {
        int man = i;
        while( man >= 0 ) {
            int wom;
            while( 1 ) {
                wom = L[man][p[man]++];
                if( R2L[wom] < 0 || R[wom][man] >
R[wom][R2L[wom]] ) break;
            }
            int hubby = R2L[wom];
            R2L[L2R[man] = wom] = man;
            man = hubby;
        }
    }
}
```

MIN COST MAX FLOW (Multiple Edge)

```
typedef struct {  int v,flow,cost,cap; }node;
node parent [ MAX_N+1 ];
int dist[ MAX_N+1 ], from[ MAX_N+1 ], v, e;
vector<node>  adj[MAX_N+1];
int augment(int s,int t) {
   bool update;
   FOR(i,1,v) dist[i] = INF,from[i]=-1;
   dist[s] = 0;
   FOR(i,1,v-1) {
     //bellman ford
      update = false;
      FOR(j,1,v) REP(k,adj[j].size())
         if (adj[j][k].flow < adj[j][k].cap)   {
            int edgeCost = adj[j][k].cost;
            if (adj[j][k].flow < 0) edgeCost = -
edgeCost;
            if (dist[adj[j][k].v] > dist[j] + edgeCost)
{
               dist[adj[j][k].v] = dist[j] + edgeCost;
               parent[adj[j][k].v] = adj[j][k];
               from[adj[j][k].v] = j;
               update = true;
            }
         }
      if (!update) break;
   }
   if (dist[t]==INF) return 0;
   int p = t,f = INF,pa;
   node temp;
   for(;from[p]!=-1;p=from[p]) {
      pa = from[p];
      temp = parent[p];
      f = min(f, temp.cap-temp.flow);
   }
   p = t;
   for(;from[p]!=-1;p=from[p])  {
      pa = from[p];
      REP(i,adj[pa].size()){
         if (adj[pa][i].v==p &&
adj[pa][i].cost==parent[p].cost){
            adj[pa][i].flow += f;
            break;
         }}
      REP(i,adj[p].size()){
         if (adj[p][i].v==pa && adj[p][i].cost ==
parent[p].cost){
            adj[p][i].flow-=f;
            break;
         }}
   }
   return f*dist[t];
}
void addedge(int v1,int v2,int c,int cap,int flow=0) {
   bool f = true;
   REP(i,adj[v1].size()) {if (adj[v1][i].v==v2 &&
adj[v1][i].cost==c) {
      adj[v1][i].cap+=cap;
      adj[v1][i].flow+=flow;
      f = false;
      break;
   }}
   if (f) adj[v1].push_back((node){v2,flow,c,cap});
}//Add undirected edge a<->b dengan cost c, kapasitas
1
addedge(a,b,c,1);
addedge(b,a,c,1);
directed edge IN(i) -> OUT(i) dengan cost a dan
kapasitas 1
addedge(IN(i),OUT(i),-a,1,0);
addedge(OUT(i),IN(i),a,1,1); //blocking flow
```

BELLMAN FORD (NEGATIVE CYCLE DETECTION)

```
int bellman_ford(int s) {
    int i, j;
    for (i = 0; i < n; ++i) d[i] = INFINITY;
    d[s] = 0;
    for (i = 0; i < n - 1; ++i)
        for (j = 0; j < e; ++j)
            if (d[edges[j].u] + edges[j].w <
d[edges[j].v])
                d[edges[j].v] = d[edges[j].u] +
edges[j].w;
    for (j = 0; j < e; ++j) // Negative detection
        if (d[edges[j].u]+edges[j].w<d[edges[j].v])
return 0;
    return 1;
}
```

RUMUS SEGITIGA BAG 2

Law of Cosines

$c^2 = a^2 + b^2 - 2*a*b*\cos C$

Area

$A = 1/2 * h * c$

$A = 1/2 * a * b * \sin C$

$A = (c^2 * \sin A * \sin B) / (2 * \sin C)$

Heron's Formula

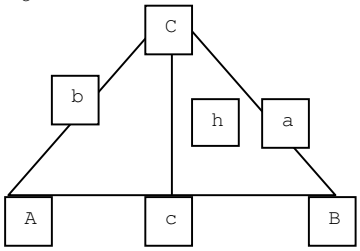
$A = \sqrt{s * s_a * s_b * s_c}$

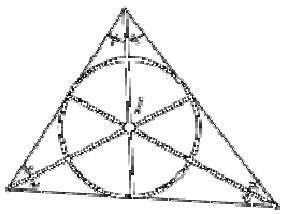
$s = 1/2 * (a+b+c)$

$s_a = s - a$

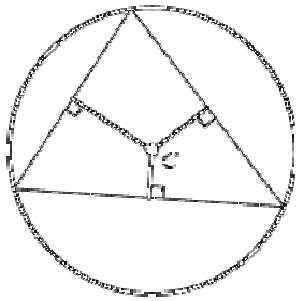
$s_b = s - b$

$s_c = s - c$





incircle



circumcircle

$$L = \tfrac{1}{2}.a.b.\sin(C)$$

$$L = \sqrt{s(s-a)(s-b)(s-c)}$$

$$L = \tfrac{abc}{4R}$$

$$L = r.s$$

$$a = \sqrt{b^2 + c^2 - 2bc.\cos(A)}$$

$$\tfrac{a}{\sin(A)} = \tfrac{b}{\sin(B)} = \tfrac{c}{\sin(C)}$$

$$m = \sqrt{\frac{2b^2 + 2c^2 - a^2}{4}}$$

Titik pusat lingkaran dalam (*inradius*) didapat dari perpotongan garis bagi segitiga (*angle bisector*), garis yang membagi sudut sama besar. Titik pusat lingkaran luar (*circumcircle*) didapat dari perpotongan garis tinggi segitiga yang membagi masing-masing sisi sama panjang (*perpendicular bisector*). Median segitiga adalah titik temu garis berat segitiga (median), garis yang membagi sisi menjadi sama panjang. dimana *m* adalah panjang garis berat (median) yang membagi sisi *a*.

MATHEMATIC FORMULA

Rumus Deret:

```
LL Sn(LL a, LL b) {
    LL range = (b - a) + 1, ret;
    ret = range * (a + b) / 2;
    return ret;
} Sn(a,b) = a + (a+1) + (a+2) + ... + b
```

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\left[(n+1)^{m+1} - 1 - \sum_{i=1}^{n}\left((i+1)^{m+1} - i^{m+1} - (m+1)i^m\right)\right]$$

$$\sum_{k=0}^{n} kCm = (n+1)C(m+1)$$

Sum of the n first triangular number is
S(n) = ((n)(n+1)(n+2))/6

One can efficiently test whether a positive integer x is a triangular number by computing
n = (sqrt(8x + 1) -1) / 2
if n is integer, then x is the nth triangular number.
if n is not an integer, then x is not triangular.

Point to line & Line to segment distance

```
// Point to line
Returns the distance from p to the line defined by {a,
b}.The closest point on the line is returned through
(cpx,cpy). Does not work for degenerate lines (when
answer is undefined).
REQUIRES: dist()
double distToLine(
    double ax, double ay,
    double bx, double by,
    double px, double py,
    double *cpx, double *cpy ) {
// Formula: cp = a + (p-a).(b-a) / |b-a| * (b-a)
double proj = ((px - ax)*(bx - ax) + (py - ay)*(by -
ay)) /((bx - ax) * (bx - ax) + (by - ay) * (by - ay));
        *cpx = ax + proj * ( bx - ax );
        *cpy = ay + proj * ( by - ay );
        return dist(px, py, *cpx, *cpy);
}
// Line to segment
Returns the distance from p to the line segment ab.
Theclosest point on ab is returned through (cpx, cpy).
Workscorrectly for degenerate line segments (a == b).
REQUIRES: dist(), distToLine()
double distToLineSegment(
    double ax, double ay,
    double bx, double by,
    double px, double py,
    double *cpx, double *cpy) {
    if ( (bx - ax)*(px - ax) + (by - ay)*(py - ay) <
EPS ) {
        *cpx = ax; *cpy = ay;
        return dist(ax, ay, px, py);
    }
    if ( (ax - bx)*(px - bx) + (ay - by)*(py - by) <
EPS ) {
        *cpx = bx; *cpy = by;
        return dist(bx, by, px, py);
    }
    return distToLine(ax, ay, bx, by, px, py, cpx,
cpy);
}
```

Lowest Common Ancestor

```
vector< pair<int,int> > v[50004];
int P[50005][17], T[50004], L[50004];
int vis[50005], dist[50005];
void process(int N) {
    for(int i=0; i<N; i++) {
        for(int k=0; (1<<k) < N; k++) P[i][k] = -1;
    }
    for(int i=0; i<N; i++) P[i][0] = T[i];
    for(int k=1; (1<<k) < N; k++) {
        for(int i=0; i<N; i++) {
            if ( P[i][k-1] != -1 ) P[i][k] = P[P[i][k-
1]][k-1];
        }
    }
}
int LCA(int A, int B) {
    if ( L[A] < L[B] ) swap(A,B);
    int log = 0;
    for(log = 1; (1<<log) <= L[A]; log++);
    log--;
    for(int i=log; i >= 0; i--) {
        if ( L[A] - ( 1<<i ) >= L[B] ) A = P[A][i];
    }
    if ( A == B ) return A;
    for(int i=log; i >= 0; i--) {
        if ( P[A][i] != -1 && P[A][i] != P[B][i]) A =
P[A][i], B = P[B][i];
    }
    return T[A];
}
void bfs(int ffirst) {
    // bfs to build L Table ( Level ), and T Table
( Previous node )
}
int main() {
    int N, M;
    while(scanf("%d", &N)!=EOF) {
        for(int i=0, f, t, w; i<N-1; i++) {
            scanf("%d %d %d", &f, &t, &w);
            v[f].push_back( make_pair( t, w ) );
            v[t].push_back( make_pair( f, w ) );
        }
```

```
        bfs( 0 );
        process(N);
        scanf("%d", &M);
        for(int i=0, f, t; i<M; i++) {
            scanf("%d %d", &f, &t);
            printf("%d\n", dist[f] + dist[t] -
2*dist[LCA(f,t)] );
        }
        for(int i=0; i<N; i++) v[i].clear();
    }
}
```

Closest Pair

```
const double inf = 10000.0;
const double eps = 1e-9;
typedef long long ll;
struct point {double x, y;};
int cmp(const point &a, const point &b) {
    return a.x < b.x;
}
inline double edist(point &a, point &b) {
    return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-
b.y));
}
inline double dabs(double a) {
    return (a < 0) ? -a : a;
}
inline double dmin(double a, double b) {
    return (a < b) ? a : b;
}
int n, buff[10005], Y[10005], Yak[10005];
point X[10005];
inline double conquer(int l,int mid,int r,double dn) {
    for (int i=l,j=mid+1,k=l; i<=mid || j<=r; k++) {
        if ((j > r) || (i<=mid && X[Y[i]].y <
X[Y[j]].y)) {
            buff[k] = Y[i], i++;
        } else {
            buff[k] = Y[j], j++;
        }
    }
    int nak = 0;
    for (int i = l; i<=r; i++) {
        Y[i] = buff[i];
        if (dabs(X[mid].x - X[Y[i]].x) <= dn) {
            Yak[nak++] = Y[i];
        }
    }
    double ret = dn;
    for (int i = 0; i < nak; i++) {
        for (int j = i+1; j < nak; j++) {
            if (dabs(X[Yak[i]].y-X[Yak[j]].y)>ret)
break;
           ret = dmin(ret,edist(X[Yak[i]], X[Yak[j]]));
        }
    }
    return ret;
}
inline double divide(int l, int r) {
    double ret = inf;
    if (l < r) {
        double dl = divide(l, (l+r)/2);
        double dr = divide((l+r)/2+1, r);
        ret = conquer(l, (l+r)/2, r, dmin(dl, dr));
    }
    return ret;
}
int main() {
    while (scanf("%d", &n) != EOF && n) {
        rep(i, n) scanf("%lf %lf", &X[i].x, &X[i].y);
        sort(X, X+n, cmp);
        rep(i, n) Y[i] = i;
        double ans = divide(0, n-1);
        if (dabs(ans-inf) <= eps) printf("INFINITY\n");
         else printf("%.4lf\n", ans);
    }
}
```

Maxflow (ADJ List)

```
typedef struct { int v,flow; }node;
vector<node> adj[MAX_N+1],cap[MAX_N+1];
bool visited[MAX_N+1];
int path[MAX_N+1], parent[MAX_N+1], v;
int bfs(int s,int t){
   bool found = 0;
   FILL(visited, 0), FILL(parent,-1);
   queue<int> q;
   q.push(s);
   visited[s] = 1;
   while (!q.empty() && !found) {
      int node = q.front();q.pop();
      for(int i=0;i<adj[node].size() && !found;i++)  {
         int temp = adj[node][i].v;
         if (!visited[temp] && adj[node][i].flow>0) {
            q.push(temp);
            visited[temp] = true;
            parent[temp] = node;
            if (temp==t) found = 1;
         }
      }
   }
   int capacity = INF;
   int nod = t;
   for(;parent[nod]!=-1;nod=parent[nod])  {
      int p = parent[nod];
      REP(i,adj[p].size()) if (adj[p][i].v==nod) {
         capacity = min(capacity, (int)adj[p][i].flow);
         break;
      }
   }
   if (capacity==INF) return 0;
   nod  = t;
   for(;parent[nod]!=-1;nod=parent[nod])   {
      int p = parent[nod];
      REP(i,adj[p].size()) if (adj[p][i].v==nod) {
         adj[p][i].flow -=capacity;
         break;
      }
      REP(i,adj[nod].size()) if
(adj[nod][i].v==parent[nod]) {
         adj[nod][i].flow+=capacity;
         break;
      }
   }
   return capacity;
}
```

Heavy Light Decomposition

```
// Ekow
typedef struct Tedge { int s,t,cost,id; }edge;
typedef struct Tnode { int L,R,value; }node;
class SegmentTree {
    private:
        vector<node> tree;
        void build(vector<edge> &data,int L,int R,int
idx) {
            tree[idx].L = L;
            tree[idx].R = R;
            if (L==R) {
                tree[idx].value = data[L].cost;
                return;
            }
            int mid = (L+R) >> 1;
            build(data,L,mid,LEFT(idx));
            build(data,mid+1,R,RIGHT(idx));
```

```
            tree[idx].value =
MAX(tree[LEFT(idx)].value,tree[RIGHT(idx)].value);
        }
    public:
    void create(vector<edge> &data)  {
        int log=0;
        while ((1<<log) < data.size()) log++;
        log++;
        tree.resize((1<<log));
        build(data,0,data.size()-1,1);
    }
    int query(int L,int R,int idx) {
        if (tree[idx].L == L && tree[idx].R == R)
return tree[idx].value;
        int mid = (tree[idx].L + tree[idx].R)>>1;
        int ret = -INF;
        if ( L <= mid) ret >?=
query(L,MIN(R,mid),LEFT(idx));
        if (R > mid) ret >?=
query(MAX(mid+1,L),R,RIGHT(idx));
        return ret;
    }
    void update(int L,int idx,int value){
        if (tree[idx].L == L && tree[idx].R == L) {
            tree[idx].value = value;
            return;
        }
        int mid = (tree[idx].L + tree[idx].R)>>1;
        if ( L <= mid) update(L,LEFT(idx),value);
        else update(L,RIGHT(idx),value);
        tree[idx].value =
MAX(tree[LEFT(idx)].value,tree[RIGHT(idx)].value);
    }
};
int P[10001][15],pathIndex[10001],pathOffset[10001];;
vector<edge> tree[10001];
vector<vector<edge> > paths;
int level[10001], treesize[10001],pid[10001],n;
edge Edge[10001];
bool hasheavy[10001], vis[10001];
vector<SegmentTree> trees;
inline int LCA(int a,int b) {
    if (level[a] > level[b]) swap(a,b);
    int diff = level[b] - level[a],log=(int)
log2(diff);
    while (diff) {
        if ((1 << log) <= diff) {
            b = P[b][log];
            diff -= (1<<log);
        }
        log--;
    }
    if (a==b) return a;
    for(int i=(int)log2(level[a]);i>=0;i--) {
        if (P[a][i] != -1 && P[a][i]!=P[b][i]) {
            a = P[a][i],b = P[b][i];
        }
    }
    return P[a][0];
}
void dfs(int x,int lev,int parent) {
    level[x] = lev;
    vis[x] = hasheavy[x] = 0;
    treesize[x]++;
    REP(i,tree[x].size()) {
        if (tree[x][i].t!=parent) {
            dfs(tree[x][i].t,lev+1,x);
            pid[tree[x][i].t] = tree[x][i].id;
            P[tree[x][i].t][0] = x;
            treesize[x]+=treesize[tree[x][i].t];
        }
    }
    REP(i,tree[x].size()) {
        if (tree[x][i].t!=parent) {
           if (treesize[tree[x][i].t]*2 >= treesize[x])
hasheavy[x] = 1;
        }
    }
}
inline void decompose() {
        vis[1] = 1;
        FOR(i,1,n) {
            int node = i;
            if (vis[node] || hasheavy[node]) continue;
            vector<edge> path;
            while (1) {
                bool isheavy = (treesize[node]*2 >=
treesize[P[node][0]]);
                vis[node] = true;
                path.push_back(Edge[pid[node]]);
                if (path[path.size()-1].s != node)
swap(path[path.size()-1].s,path[path.size()-1].t);
                node = P[node][0];
                if (vis[node]) break;
                if (!isheavy) break;
            }
            paths.push_back(path);
        }
        trees.clear();
        trees.resize(paths.size());
        int id;
        REP(i,paths.size()) {
            trees[i].create(paths[i]);
            REP(j,paths[i].size()) {
                id = paths[i][j].id;
                pathIndex[id] = i;
                pathOffset[id] = j;
            }
        }
}
inline int query(int a, int ancestor) {
    int diff = level[a] - level[ancestor];
    int ret = -INF,to;
    if (pid[a]==-1) return ret;
    int treeid = pathIndex[pid[a]], offset =
pathOffset[pid[a]];
    while (diff) {
        to = MIN( paths[treeid].size()-1 , offset +
diff -1) ;
        ret >?= trees[treeid].query(offset,to,1);
        diff -= (to-offset+1);
        a = paths[treeid][paths[treeid].size()-1].t;
        if (pid[a] == -1) break;
        treeid = pathIndex[pid[a]], offset =
pathOffset[pid[a]];
    }
    return ret;
}
int main() {
    int t,a,b,c,parent;
    int idx,offset,temp;
    char cmd[20];
    pid[1] = -1;
    for(scanf("%d",&t);t--;) {
        FILL(P,-1);
        //FILL(vis,0);
        //FILL(hasheavy,0);
        paths.clear();
        FILL(treesize,0);
        scanf("%d",&n);
        REP(i,n+1) tree[i].clear();
        REP(i,n-1) {
            scanf("%d %d %d",&a,&b,&c);
            Edge[i+1] = (edge){a,b,c,i+1};
```

```
            tree[a].push_back((edge){a,b,c,i+1});
            tree[b].push_back((edge){b,a,c,i+1});
        }
        dfs(1,0,-1);
        for(int pang=1,val=2;val<=n;pang++,val<<=1) {
            FOR(j,1,n) {
                parent = P[j][pang-1];
                if (parent!=-1) P[j][pang] =
P[parent][pang-1];
            }
        }
        decompose();
        while (scanf("%s",cmd)) {
            if (cmd[0]=='D') break;
            scanf("%d %d",&a,&b);
            if (cmd[0]=='C') {
                idx = pathIndex[a];
                offset = pathOffset[a];
                trees[idx].update(offset,1,b);
            } else {
                temp = LCA(a,b);

    printf("%d\n",MAX(query(b,temp),query(a,temp)));
            }
        }
    }
    return 0;
}
// Shu, soal kmrn
const int maxn = 50005;
vector <int> v[maxn];
vector <vector<int> > paths;
int s[maxn], loc[maxn], pos[maxn];
int par[maxn], head[maxn];
int count_size(int x, int p = -1) {
    s[x]   = 1;
    par[x] = p;
    REP(i,v[x].size()) if ( v[x][i] != p )
        s[x] += count_size(v[x][i],x);
    return s[x];
}
void decompose(int x, int p, int id) {
    paths[id].push_back(x);
    loc[x] = id;
    pos[x] = paths[id].size() - 1;
    REP(i,v[x].size()) if ( v[x][i] != p ) {
        if ( s[v[x][i]] * 2 >= s[x] )
decompose(v[x][i],x,id);
        else {
            paths.push_back(vector<int>(0));
            head[paths.size()-1] = v[x][i];
            decompose(v[x][i],x,paths.size()-1);
        }
    }
}
vector <pii> range;
int idx[maxn];
int rmq[131072][17];
void euler_tour(int x, int p, int depth) {
    idx[x] = range.size();
    range.push_back(pii(x,depth));
    REP(i,v[x].size()) if ( v[x][i] != p ) {
        euler_tour(v[x][i],x,depth+1);
        range.push_back(pii(x,depth));
    }
}
void compute_rmq() {
    int n = range.size();
    REP(i,range.size()) rmq[i][0] = i;
    for ( int j = 1; 1 << j <= n; j++ )
        for ( int i = 0; i + (1 << j) - 1 < n; i++ )
            if ( range[rmq[i][j-1]].second <
range[rmq[i+(1<<(j-1))][j-1]].second )
                rmq[i][j] = rmq[i][j-1];
            else
                rmq[i][j] = rmq[i+(1<<(j-1))][j-1];
}
int lca(int a, int b) {
    int L = idx[a], R = idx[b];
    if ( L > R ) swap(L,R);
    int k = (int)floor(log2(R-L+1));
    if ( range[rmq[L][k]].second <= range[rmq[R-
(1<<k)+1][k]].second )
        return range[rmq[L][k]].first;
    return range[rmq[R-(1<<k)+1][k]].first;
}
struct tdata { int L, R, mid, val; };
inline int LEFT(int x) { return x << 1; }
inline int RIGHT(int x) { return (x << 1) + 1; }
vector <vector<tdata> > tree;
void build(int id, int L, int R, int x) {
    tree[id][x].L   = L;
    tree[id][x].R   = R;
    tree[id][x].mid = (L + R) >> 1;
    tree[id][x].val = 0;
    if ( L != R ) {
        build(id,L,tree[id][x].mid,LEFT(x));
        build(id,tree[id][x].mid+1,R,RIGHT(x));
    }
}
void add(int id, int L, int R, int x, int c) {
    if ( L > R ) return;
    if ( tree[id][x].L == L && tree[id][x].R == R )
tree[id][x].val += c;
    else {
        add(id,L,min(tree[id][x].mid,R),LEFT(x),c);
        add(id,max(L,tree[id][x].mid+1),R,RIGHT(x),c);
    }
}
int query(int id, int t, int x) {
    int ret = tree[id][x].val;
    if ( tree[id][x].L != tree[id][x].R ) {
        int next = (t<=tree[id][x].mid) ? LEFT(x) :
RIGHT(x);
        ret += query(id,t,next);
    }
    return ret;
}
void insert(int a, int b, int c) {
    int x = a;
    int id = loc[x];
    while ( id != loc[b] ) {

    add(id,min(pos[x],pos[head[id]]),max(pos[x],pos[he
ad[id]]),1,c);
        x = par[head[id]];
        id = loc[x];
    }
    add(loc[b],min(pos[x],pos[b]),max(pos[x],pos[b]),1
,c);
}

int main(){
    int T;
    scanf( "%d", &T );
    for ( int tc = 1; tc <= T; tc++ ) {
        printf( "Case #%d:\n", tc );
        int n, a, b, c, q;
        scanf( "%d", &n );
        REP(i,n) v[i].clear();
        REP(i,n-1) {
            scanf( "%d %d", &a, &b );
            v[a].push_back(b);
```

```
            v[b].push_back(a);
        }
        count_size(0);
        range.clear();
        memset(idx,0,sizeof(idx));
        euler_tour(0,0,0);
        compute_rmq();
        paths.clear();
        paths.push_back(vector<int>(0));
        head[0] = 0;
        decompose(0,0,0);
        tree.clear();
        tree.resize(paths.size());
        REP(i,paths.size()) {
            int t = 1;
            while ( t < paths[i].size() ) t <<= 1;
            t <<= 1;
            tree[i].resize(t);
            build(i,0,(t>>1)-1,1);
        }
        scanf( "%d", &q );
        REP(i,q) {
            scanf( "%d %d %d", &a, &b, &c );
            int p = lca(a,b);
            insert(a,p,c);
            insert(b,p,c);
            insert(p,p,-c);
        }
        REP(i,n) printf( "%d\n",
query(loc[i],pos[i],1) );
    }
}
```

Enhanced suffix Array

```
#define MAX_N 100000
typedef struct  {  int a[2],idx; }ttemp;
inline bool cmp(const ttemp &a,const ttemp &b) {
   return (a.a[0]==b.a[0]) ? (a.a[1] < b.a[1]) :
(a.a[0] < b.a[0]);
}
ttemp L[MAX_N],temp[MAX_N];
int step,p[20][MAX_N],lt,lq, a[MAX_N+1];
inline void rasort(ttemp x[],int n) {
   int k=-1;
   memset(a,0,sizeof(a));
   REP(i,n) a[x[i].a[1]+1]++, k=max(k,x[i].a[1]+1);
   for(int i=1;i<=k;i++) a[i]+=a[i-1];
   for(int i=n-1;i>=0;i--) temp[--a[x[i].a[1]+1]] =
x[i];
   memset(a,0,sizeof(a));
   k = -1;
   REP(i,n) {a[temp[i].a[0]+1]++;
k=max(k,temp[i].a[0]+1);}
   for(int i=1;i<=k;i++) a[i]+=a[i-1];
   for(int i=n-1;i>=0;i--) x[--a[temp[i].a[0]+1]] =
temp[i];
}
inline void generate(char s[],int p[][MAX_N]) {
   int cnt;
   for(int i=0;i<lt;i++) p[0][i] = s[i];
   for( step=1,cnt=1;(cnt>>1) < lt; step++,cnt<<=1) {
      for(int i=0;i<lt;i++)  {
         L[i].a[0] = p[step-1][i];
         L[i].a[1] = ( i + cnt < lt ? p[step-1][i +
cnt] : -1 );
         L[i].idx = i;
      }
      rasort(L,lt);
      //sort(L,L+lt);  n log^2 n
      for(int i=0;i<lt;i++)
         p[step][L[i].idx] = i>0 && L[i].a[0] == L[i-
1].a[0] && L[i].a[1] == L[i-1].a[1] ? p[step][L[i-
1].idx]:i ;
   }
   step--;
}
char pattern[1100],text[100010];
//compare pattern suffix[idx]
inline int compare(int idx) {
   int i = 0;
   for(;i<lq && idx < lt;i++,idx++) {
      if (pattern[i] > text[idx]) return -1;
      if (pattern[i] < text[idx]) return 1;
   }
   return (i == lq ? 0 : -1 );
}
inline int lcp(int x,int y) {
   if (x==y) return lt-x;
   int res = 0;
   for(int k = step-1; k>=0 && x<lt && y<lt; k--)
      if (p[k][x]==p[k][y]){
         x+=(1<<k),y+=(1<<k),res+=(1<<k);}
   return res;
}
//cari many pattern pada text yang panjang, preprocess
text
int L = 0,R = lt-1,mid, found = false;
while ( L <= R && !found) {
      mid = (L+R)>>1;
      int temp = compare(arr[mid]);
      if (temp==0) found = 1;
      else if (temp==1) R = mid-1;
      else L = mid+1;
}
//Longest palindrome
S = S + rev(S), edit LCP
for(int i=0;i<lt/2-1 ;i++){
   res >?= lcp(lt-i-1,i+1);
   if (i)  res>?= 1 + lcp(lt-i,i+1);
}
//Distinct substring
if (lt&1) res = ((lt+1)>>1)*lt; else res = (lt>>1) *
(lt+1);
REP(i,lt) arr[p[step-1][i]] = i;
REP(i,lt-1)  res -= lcp(arr[i],arr[i+1]);
//longest palindrome, maximum K change
inline int longpal(int left,int right,int k){
   int temp = 0,a;
   for(int j=0;j<=k;){
      a = lcp(right,left);
      temp +=2*a;
      left +=a;
      right+=a;
      if (a==0)  {
         if (j==k) break;
         temp+=2;
         left++,right++;
         j++;
      }
      if (left>= lt || right >= lt/2) break;
   }
   return temp;
}
```

Polygon diameter

```
Polygon struct Point {
    double x, y;
    Point(double xx, double yy) {
        x = xx;
        y = yy;
    }
    Point() {}
    void print() {
        cout << "(" << x << "," << y << ")";
    }
};
```

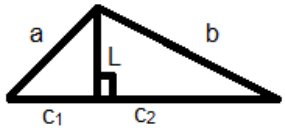
```
double area(Point &p1, Point &p2, Point &p3) {
    return 0.5 * fabs((p1.x - p2.x) * (p1.y - p3.y) -
(p1.x - p3.x) * (p1.y - p2.y));
}
double len(double dx, double dy) {
    return sqrt(dx * dx + dy * dy);
}
double distance2Point(Point &p1, Point &p2) {
    return len(p1.x - p2.x, p1.y - p2.y);
}
struct ConvexPolygon {
    Point *p;
    int n;
    ConvexPolygon(Point *pp, int nn) {
        p = pp; n = nn;
    }
    double diameter() {
        int a, b;
        double d;
        double area1, area2;
        d = 0;
        if(n == 2) return distance2Point(p[0], p[1]);
        if(n == 3) return max(distance2Point(p[0],
p[1]),
              max(distance2Point(p[1], p[2]),
distance2Point(p[0], p[2])));
        a = n - 1; b = 1;
        area1 = area(p[a], p[(a + 1) % n], p[(b + 1) %
n]);
        area2 = area(p[a], p[(a + 1) % n], p[b]);
        while(area1 > area2) {
            b = (b + 1) % n;
            area1 = area(p[a], p[(a + 1) % n], p[(b +
1) % n]);
            area2 = area(p[a], p[(a + 1) % n], p[b]);
        }
        while(b != 0) {
            a = (a + 1) % n;
            d = max(d, distance2Point(p[a], p[b]));
            area1 = area(p[a], p[(a + 1) % n], p[(b +
1) % n]);
            area2 = area(p[a], p[(a + 1) % n], p[b]);
            while(area1 > area2) {
                b = (b + 1) % n;
                area1 = area(p[a], p[(a + 1) % n], p[(b
+ 1) % n]);
                area2 = area(p[a], p[(a + 1) % n],
p[b]);
                if(b == 0) return d;
                d = max(d, distance2Point(p[a], p[b]));
            }
        }
        return d;
    }
};
```

Polygon Area

```
int area = 0;
int N = lengthof(p);
for(int i = 1; i+1<N; i++){
    int x1 = p[i][0] - p[0][0];
    int y1 = p[i][1] - p[0][1];
    int x2 = p[i+1][0] - p[0][0];
    int y2 = p[i+1][1] - p[0][1];
    int cross = x1*y2 - x2*y1;
    area += cross;
}
return abs(cross/2.0);
```

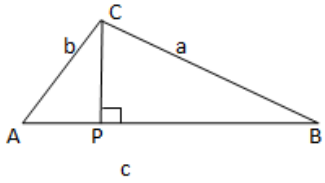
Misc.

Dalil Steward



$L^2 \cdot c = a^2.c_2 + b^2.c_1 - c_1.c_2.c$

Dalil Proyeksi



$AP = (b^2 + c^2 - a^2) / 2c$

Centroid of Polygon !

The centroid of a non-self intersecting closed polygon defined by n vertices (x0,y0), ..., (x(n-1),y(n-1)), is the point (Cx, Cy), where

$$C_x = \frac{1}{6A}\sum_{i=0}^{n-1}(x_i + x_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

$$C_y = \frac{1}{6A}\sum_{i=0}^{n-1}(y_i + y_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

and where A is the polygon's signed area,

$$A = \frac{1}{2}\sum_{i=0}^{n-1}(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

SARRUS

$$\sum_{i=0}^{n}(X_i * Y_{i+1} * Z_{i+2}) - (X_{i+2} * Y_{i+1} * Z_i)$$

LINE SEGMENT INTERSECTION POINT

$$Y = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & y_1 - y_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & y_3 - y_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}},$$

$$X = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & x_1 - x_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & x_3 - x_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}}$$

Segment Tree

Frequent Values

```
typedef struct {int a, b, Lm, Rm, Mm;} st;
void init( int idx, int a, int b ) {
    v[idx].a = a, v[idx].b = b;
    if ( a == b ) {
```

```
        v[idx].Lm = v[idx].Mm = v[idx].Rm = 1;
        return;
    }
    int mid = ( a + b ) / 2;
    init( 2*idx, a, mid );
    init( 2*idx+1, mid+1, b );
    v[idx].Mm = max( v[2*idx].Mm, v[2*idx+1].Mm );
    if ( arr[mid] == arr[mid+1] ) v[idx].Mm =
max( v[idx].Mm, v[2*idx].Rm + v[2*idx+1].Lm );
    v[idx].Lm = v[2*idx].Lm;
    if ( arr[a] == arr[mid+1] ) v[idx].Lm +=
v[2*idx+1].Lm;
    v[idx].Rm = v[2*idx+1].Rm;
    if ( arr[mid] == arr[b] ) v[idx].Rm += v[2*idx].Rm;
}
```

SUM in RANGE (LAZY PROPAGATION)

```
typedef struct {int a,b;LL add, nsum;}st;
st v[300000];
inline void lzPro( int idx, LL c ) {
    if ( v[idx].a != v[idx].b ) v[idx].add += c;
    v[idx].nsum = v[idx].nsum + ( LL( LL(v[idx].b) -
LL(v[idx].a) + 1LL ) * c );
}
inline void PushPro( int idx ) { // Push Propagation...
    if ( v[idx].add == 0LL ) return;
    lzPro( 2*idx, v[idx].add );
    lzPro(2*idx+1,v[idx].add );
    v[idx].add = 0LL; // done
}
void add( int idx, int a, int b, LL c ) {
    if ( v[idx].a == a && v[idx].b == b ) {
        lzPro( idx, c );
        return;
    }
    PushPro( idx ); // push propagation if ada.... :D
    int mid = ( v[idx].a + v[idx].b ) / 2;
    if ( v[idx].a <= a && b <= mid ) {
        add( 2*idx, a, b, c );
    } else if ( mid < a && b <= v[idx].b ) {
        add( 2*idx+1, a, b, c );
    } else {
        add( 2*idx, a, mid, c );
        add(2*idx+1,mid+1,b,c );
    }
    v[idx].nsum = v[2*idx].nsum + v[2*idx+1].nsum;
}
LL retSum( int idx, int a, int b ) {
    if ( v[idx].a == a && v[idx].b == b ) return
v[idx].nsum;
    PushPro( idx );
    int mid = ( v[idx].a + v[idx].b ) / 2;
    if ( v[idx].a <= a && b <= mid ) return
retSum( 2*idx, a, b );
    else if ( mid < a && b <= v[idx].b ) return
retSum(2*idx+1, a, b );
    else return retSum(2*idx,a,mid) +
retSum(2*idx+1,mid+1,b);
}
```

BRACKETS ! (Partial SUM Cannot Negative)

```
typedef struct {int a, b, min, sum;}nt;
char s[33000];
nt v[100000];
void init( int idx, int a, int b ) {
    v[idx].a = a, v[idx].b = b;
    if ( a == b ) {
        v[idx].min = v[idx].sum =(s[a-1]=='(')?1:-1;
        return;
    }
    int mid = ( a + b ) / 2;
    init( 2*idx, a, mid );
    init(2*idx+1,mid+1,b);
    v[idx].sum = v[2*idx].sum + v[2*idx+1].sum;
    if ( v[2*idx+1].min + v[2*idx].sum < v[2*idx].min )
{
        v[idx].min = v[2*idx+1].min + v[2*idx].sum;
    } else v[idx].min = v[2*idx].min;
}
```

KTH in RANGE

```
use upper_bound !
```

BIG INTEGER

```
const int maxlength=1000;
class bigint{
public:
    int oper,length,a[maxlength];
    bigint(int=0);
    ~bigint();
    int max(int a,int b);
    void check();
    void operator=(bigint m);
    void operator=(int m);
    void operator=(char *s);
    bool operator<(bigint m);
    bool operator<=(bigint m);
    bool operator>(bigint m);
    bool operator>=(bigint m);
    bool operator==(bigint m);
    bool operator!=(bigint m);
    bigint operator-();
    bigint operator+(bigint m);
    void operator+=(bigint m);
    bigint operator-(bigint m);
    void operator-=(bigint m);
    bigint operator*(bigint m);
    bigint operator*(int m);
    void operator*=(bigint m);
    void operator*=(int m);
    bigint operator/(bigint m);
    bigint operator/(int m);
    void operator/=(bigint m);
    void operator/=(int m);
    bigint operator%(bigint m);
    bigint operator%(int m);
    void operator%=(bigint m);
    void operator%=(int m);
};
bigint abs(bigint m);
bool read(bigint &m);
void write(bigint m);
void swrite(char *s,bigint m);
void writeln(bigint m);
bigint sqr(bigint m);
bigint sqrt(bigint m);
bigint gcd(bigint a,bigint b);
bigint lcm(bigint a,bigint b);
int bigint::max(int a,int b){return (a>b)?a:b;}
bigint::bigint(int v){
    (*this)=v;
    this->check();
}
bigint::~bigint(){}
void bigint::check(){
    for (;length>0 && a[length]==0;length--);
    if (length==0)oper=1;
}
void bigint::operator=(bigint m){
    oper=m.oper;
    length=m.length;
    memcpy(a,m.a,maxlength*sizeof(int));
    this->check();
}
void bigint::operator=(int m){
    oper=(m>0)?1:-1;
    m=abs(m);
    memset(a,0,maxlength*sizeof(int));
```

```
    for (length=0;m>0;m=m/10000){
        a[++length]=m%10000;}
    this->check();
}
void bigint::operator=(char *s){
    int i,L;
    (*this)=0;
    if (s[0]=='-' || s[0]=='+') {
        if (s[0]=='-')oper=-1;
        L=strlen(s);
        for (i=0;i<L;i++) s[i]=s[i+1];
    }
    L=strlen(s);
    length=(L+3)/4;
    for (i=0;i<L;i++) {
        a[(L-i+3)/4]=a[(L-i+3)/4]*10+(s[i]-48);}
    this->check();
}
bool bigint::operator<(bigint m) {
    if (oper!=m.oper) return oper<m.oper;
    if (length!=m.length){
        return oper*length<m.length*oper;}
    for (int i=length;i>=1;i--){
        if (a[i]!=m.a[i]){
            return a[i]*oper<m.a[i]*oper;}}
    return false;
}
bool bigint::operator<=(bigint m){return !(m<(*this));}
bool bigint::operator>(bigint m){return m<(*this);}
bool bigint::operator>=(bigint m){return !((*this)<m);}
bool bigint::operator==(bigint m){return (!((*this)<m))
&& (!(m<(*this)));}
bool bigint::operator!=(bigint m){return ((*this)<m)
|| (m<(*this));}
bigint bigint::operator-(){
    bigint c=(*this);
    c.oper=-c.oper;
    c.check();
    return c;
}
bigint abs(bigint m){
    bigint c=m;
    c.oper=abs(c.oper);
    c.check();
    return c;
}
bigint bigint::operator+(bigint m){
    if (m.length==0)return (*this);
    if (length==0)return m;
    if (oper==m.oper){
        bigint c;
        c.oper=oper;
        c.length=max(length,m.length)+1;
        for (int i=1,temp=0;i<=c.length;i++) {
        c.a[i]=(temp=(temp/10000+a[i]+m.a[i]))%10000;}
        c.check();
        return c;
    }
    return (*this)-(-m);
}
bigint bigint::operator-(bigint m){
    if (m.length==0)return (*this);
    if (length==0)return (-m);
    if (oper==m.oper){
        bigint c;
        if (abs(*this)>=abs(m)){
            c.oper=oper;
            c.length=length;
            for (int i=1,temp=0;i<=length;i++){
                c.a[i]=((temp=(-int(temp<0)+a[i]-
m.a[i]))+10000)%10000;}
            c.check();
            return c;
        }
        return -(m-(*this));
    }
    return (*this)+(-m);
}
bool read(bigint &m){
    char s[maxlength*4+10];
    if (scanf("%s",&s)==-1)return false;
    for (int i=0;s[i];i++) {if (!(s[i]>='0' &&
s[i]<='9' || (s[i]=='+' || s[i]=='-') && i==0)){
            return false;}}
    m=s;
    return true;
}
void swrite(char *s,bigint m){
    int L=0;
    if (m.oper==-1) s[L++]='-';
    sprintf(s+L,"%d",m.a[m.length]);
    for (;s[L]!=0;L++);
    for (int i=m.length-1;i>=1;i--){
        sprintf(s+L,"%04d",m.a[i]);
        L+=4;
    }
    s[L]=0;
}
void write(bigint m){
    if (m.oper==-1)printf("-");
    printf("%d",m.a[m.length]);
    for (int i=m.length-1;i>=1;i--){
        printf("%04d",m.a[i]);}
}
void writeln(bigint m){
    write(m);printf("\n");
}
bigint bigint::operator*(bigint m){
    bigint c;
    c.oper=oper*m.oper;
    c.length=length+m.length;
    for (int i=1;i<=m.length;i++){
        int number=m.a[i],j,temp=0;
        for (j=1;j<=length;j++){
            c.a[i+j-1]+=number*a[j];}
        if (i%10==0 || i==m.length){
            for (j=1;j<=c.length;j++){
    c.a[j]=(temp=(temp/10000)+c.a[j])%10000;}}
    }
    c.check();
    return c;
}
bigint bigint::operator*(int m){
    if (m<0)return -((*this)*(-m));
    if (m>100000)return (*this)*bigint(m);
    bigint c;
    c.length=length+2;
    c.oper=oper;
    int t=0;
    for (int i=1;i<=c.length;i++){
        c.a[i]=(t=(t/10000+a[i]*m))%10000;}
    c.check();
    return c;
}
bigint bigint::operator/(bigint m){
    if (m.length==0){
        printf("Division by zero.\n");
        exit(0);}
    if (abs(*this)<abs(m))return 0;
    bigint c,left;
    c.oper=oper/m.oper;
    m.oper=1;
    c.length=length-m.length+1;
    left.length=m.length-1;
```

```
    memcpy(left.a+1,a+length-
left.length+1,left.length*sizeof(int));
    for (int i=c.length;i>=1;i--){
        left=left*10000+a[i];
        int head=0,tail=10000,mid;
        while (head+1<tail){
            mid=(head+tail)/2;
            if (m*mid<=left)head=mid;
            else tail=mid;
        }
        c.a[i]=head;
        left-=m*head;
    }
    c.check();
    return c;
}
bigint bigint::operator/(int m){
    if (m<0)return -((*this)/(-m));
    if (m>100000)return (*this)/bigint(m);
    bigint c;
    c.oper=oper;
    c.length=length;
    int t=0;
    for (int i=c.length;i>=1;i--){
        c.a[i]=(t=(t%m*10000+a[i]))/m;}
    c.check();
    return c;
}
bigint bigint::operator %(bigint m){return (*this)-
((*this)/m)*m;}
bigint bigint::operator%(int m){
    if (m<0)return -((*this)%(-m));
    if (m>100000)return (*this)%bigint(m);
    int t=0;
    for (int i=length;i>=1;i--){
        t=(t*10000+a[i])%m;}
    return t;
}
bigint sqr(bigint m){return m*m;}
bigint sqrt(bigint m){
    if (m.oper<0 || m.length==0)return 0;
    bigint c,last,now,templast;
    c.length=(m.length+1)/2;
    c.a[c.length]=int(sqrt((double)m.a[c.length*2]*100
000+m.a[c.length*2-1])+1e-6);
    templast.length=c.length*2;
    templast.a[c.length*2-
1]=(c.a[c.length]*c.a[c.length])%10000;
    templast.a[c.length*2]=(c.a[c.length]*c.a[c.length
])/10000;
    templast.check();
    for (int i=c.length-1;i>=1;i--){
        last=templast;
        int head=0,tail=10000,mid,j,temp;
        while (head+1<tail){
            mid=(head+tail)/2;
            now=last;
            now.a[2*i-1]+=mid*mid;
            for (j=i+1;j<=c.length;j++){
                now.a[i+j-1]+=mid*c.a[j]*2;}
            now.length++;
            for (j=2*i-1,temp=0;j<=now.length;j++){
        now.a[j]=(temp=(temp/10000+now.a[j]))%10000;}
            now.check();
            if (now<=m){
                templast=now;
                head=mid;
            }else tail=mid;
        }
        c.a[i]=head;
    }
    c.check();
    return c;
}
void bigint::operator+=(bigint m){(*this)=(*this)+m;}
void bigint::operator-=(bigint m){(*this)=(*this)-m;}
void bigint::operator*=(bigint m){(*this)=(*this)*m;}
void bigint::operator/=(bigint m){(*this)=(*this)/m;}
void bigint::operator%=(bigint m){(*this)=(*this)%m;}
void bigint::operator*=(int m){(*this)=(*this)*m;}
void bigint::operator/=(int m){(*this)=(*this)/m;}
void bigint::operator%=(int m){(*this)=(*this)%m;}
```

CSOD

```
LL sum(LL a,LL b) {
    a--;return (b*(b+1)>>1) - (a*(a+1)>>1);
}
LL CSOD(LL n) {
    LL i,m=n>>1;
    LL out2=0,j=sqrt(n);
    for(int i=2;i<=j;i++) out2=(n/i)*i;
    for(i--;j>=1;j--) {
        out2 += (j*sum(i+1,n/j));
        i = (n/j);
    }
    out2 -= ((n*(n+1)>>1)-1);
    return out2;
}
```

CRT dan kawan2

```
Assume that f(0) = 1 and 0^0=1. f(n) = (n%10)^f(n/10)
for all n bigger than zero. Please calculate f(n)%m.
(2 <= n , m <= 10^9, x^y means the y th power of x).
LL inverse(LL a,LL m) {
    return mPow(a,phi(m)-1,m);
}
LL CRT( vector< pair<LL,int> > data ) {
    LL M = 1;
    FOREACH(it,data) M *= it->ND;
    LL ret = 0LL;
    FOREACH(it,data) {
        LL Mi = M / it->ND;
        LL temp = it->ST * Mi * inverse(Mi,it->ND);
        ret += temp;
        ret %= M;
    }
    return ret;
}

int nums[16], n;
const int LIM=1<<10;
LL brute(LL base,int pos, LL M) {
    if(pos==n) return base%M;
    int j = n;
    LL pj = 1;
    while(j-1 >= pos) {
        LL k = 1;
        REP(loop,pj) {
            k*=nums[j-1];
            if(k >= LIM) return 0;
        }
        pj = k; j--;
    }
    return mPow(base,pj,M);
}
LL solve(int,LL);
LL doit(int,int);
pair<LL,int> solve2(int pos, LL M) {
    LL a = nums[pos];
    if(GCD(a,M)==1) {
        LL b = solve( pos+1, phi((int)M) );
        return MP( mPow(a,b,M), M );
    }
    return MP( brute(a, pos+1, M), M);
}
```

```
LL solve(int pos,LL M) {
    if(M==1) return 0;
    if(pos==n) return 1;
    if(nums[pos]==1LL) return a%M;
    if(nums[pos]==0LL) return 0;
    LL m = M;
    vector< pair<LL,int> > data;
    for(LL i=2;i*i<=m;i++) {
        if(m%i!=0) continue;
        LL p=1;
        while(m%i==0) { m/=i; p*=i; }
        data.PB( solve2(pos, p) );
    }
    if(m>1) data.PB( solve2(pos, m) );
    return CRT(data);
}
void normalize() {
    nums[n] = 1;
    int newN = n;
    REP(i,n) {
        if(nums[i]==1) {
            newN = i;
            break;
        }else if(nums[i]==0) {
            int ctr0 = 0;
            while(nums[i+ctr0]==0) ctr0++;
            if(ctr0 % 2 == 0) {
                nums[i] = 1;
                newN = i;
            } else {
                if(i-1>=0) {
                    nums[i-1]=1;
                    newN=i-1;
                }
            }
            break;
        }
    }
    n=newN;
    if(nums[n-1]!=1) nums[n++]=1;
}
LL doit(int x,int m) {
    n=0;
    while(x) {
        int j=x%10;
        nums[n++]=j;
        x/=10;
    }
    normalize();
    return solve(0,m);
}
int main() {
    int x, m;
    REP(ncase,getint()) {
        scanf("%d %d", &x, &m);
        printf("%d\n",(int)doit(x,m));
    }
    return 0;
}
```

Range Sieve

```
void sieve(int L,int U,vector<int>&primes) {
    memset( isnp, 0, sizeof(isnp) );
    int nto = U - L + 1;
    for(int i=0; i<primes.size(); i++) {
        int nP = primes[i];
        if ( nP > U / nP ) break;
        int fnum = (L / nP) * nP;
        if ( fnum < L ) fnum += nP;
        if ( fnum == nP ) fnum += nP;
        for(int k=fnum-L;k<nto; k += nP ) isnp[k] = 1;
    }
    if ( L <= 1 ) isnp[1-L] = 1;
    if ( L <= 0 ) isnp[0-L] = 1;}
```

Square Collision

```
((x1>=x1t && x1<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x2>=x1t && x2<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x1<=x1t && x2>=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t)))
```

MIN CUT (Stoer Wagner)

```
int m[600][600], node[600];
int inset[600], d[600];
int reduce( int n ) {
    memset( inset, 0, sizeof(inset) );
    memset( d, 0, sizeof(d) );
    inset[ node[0] ] = 1;
    for( int i=1; i<n; i++ ) d[node[i]] =
m[node[0]][node[i]];
    int ltnode = 0;
    for( int _=1; _<n; _++ ) {
        int nd = -1;
        FOR(i, 1, n-1) {
            if ( inset[ node[i] ] ) continue;
            if ( nd == -1 ) nd = i;
            if ( d[ node[i] ] > d[ node[nd] ] ) nd = i;
        }
        inset[ node[nd] ] = 1;
        if ( _ == n -1 ) {
            REP(i,n) {
                m[ node[ltnode] ][ node[i] ] +=
m[ node[nd] ][ node[i] ];
                m[ node[i] ][ node[ltnode] ] =
m[ node[ltnode] ][ node[i] ];
            }
            int ret = d[ node[nd] ];
            node[nd] = node[n-1];
            return ret;
        }
        ltnode = nd;
        REP(i,n) {
            if ( inset[ node[i] ] ) continue;
            d[ node[i] ] += m[ node[nd] ][ node[i] ];
        }
    }
}
int mincut( int n ) {
    for( int i=0; i<n; i++ ) node[i] = i;
    int ans = ( 1 << 30 );
    while( n > 1 ) {
        int mcut = reduce( n );
        if ( mcut < ans ) ans = mcut;
        n--;
    }
    return ans;
}
int main() {
    int N, M;
    while( scanf("%d %d", &N, &M) != EOF ) {
        memset( m, 0, sizeof(m) );
        for( int i=0, a, b, c; i<M; i++ ) {
            scanf("%d %d %d", &a, &b, &c);
            m[a][b] += c;
            m[b][a] += c;
        }
        printf("%d\n", mincut( N ) );
    }}
```

Eulerian Tour Checking

```
// Every edge is visited exactly once
int N, M, d[250], di[250];
int m[205][205], pre[205];
int vis[205];
```

```
int flow( int f, int si) {
    queue< int > q;
    q.push( f );
    memset( vis, 0, sizeof(vis) );
    memset( pre, -1, sizeof(pre) );
    while( !q.empty() ) {
        int now = q.front();
        q.pop();
        if ( vis[now] == 2 ) continue;
        vis[now] = 2;
        if ( now == si ) break;
        for( int i=1; i<=N+1; i++ ) {
            int next = i;
            if ( vis[next] ) continue;
            if ( m[now][next] > 0 ) {
                vis[next] = 1;
                pre[next] = now;
                q.push( next );
            }
        }
    }
    int nt = si, ret = ( 1 << 30 );
    while( pre[nt] != -1 ) {
        int bf = pre[nt];
        ret = min( ret, m[bf][nt] );
        nt = bf;
    }
    if ( ret >= ( 1 << 30 ) ) return 0;
    nt = si;
    while( pre[nt] != -1 ) {
        int bf = pre[nt];
        m[bf][nt] -= ret;
        m[nt][bf] += ret;
        nt = bf;
    }
    return ret;
}

int main() {
    int ntc;
    scanf("%d", &ntc);
    while( ntc-- ) {
        scanf("%d %d", &N, &M);
        memset( d, 0, sizeof(d) );
        memset( m, 0, sizeof(m) );
        for( int i=0, a, b, c; i<M; i++ ) {
            scanf("%d %d %d", &a, &b, &c);
            d[a]++, d[b]--;
            if ( !c ) m[a][b]++;
        }
        int can = 1;
        for( int i=1; can && i<=N; i++ ) {
            int x = ( d[i] < 0 ) ? -d[i] : d[i];
            if ( x%2 == 1 ) can = 0;
            d[i] = d[i] / 2;
        }
        if ( can ) {
            int tflow = 0;
            for( int i=1; i<=N; i++ ) {
                if ( d[i] < 0 ) m[i][N+1] = -d[i];
                else if ( d[i] > 0 ) {
                    m[0][i] = d[i];
                    tflow += d[i];
                }
            }
            int tans, ans = 0;
            while( (tans = flow( 0, N+1) ) > 0 ) ans +=
tans;
            can = can && ( ans == tflow );
        }
        puts( (can) ? "possible" : "impossible" );
    }
    return 0;
}
```

SCC

```
int N, M;
vector<int> v[1003];
int vis[1003], in[1003];
int low[1003], idx[1003], inStack[1003], comp[1003];
int ntime, nComp;
stack<int> stScc;
int tarjan( int x ) {
    low[x] = idx[x] = ntime++;
    stScc.push( x );
    inStack[x] = 1;
    for( int i = 0; i < v[x].size(); i++ ) {
        if ( !idx[ v[x][i] ] ) {
            tarjan( v[x][i] );
            low[x] = min(low[x], low[v[x][i]]);
        } else if ( inStack[ v[x][i] ] ) {
            low[x] = min( low[x], idx[ v[x][i] ] );
        }
    }
    if ( low[x] == idx[x] ) {
        int nx;
        do {
            nx = stScc.top();
            stScc.pop();
            inStack[nx] = 0;
            comp[nx] = nComp;
        } while ( nx != x );
        nComp++;
    }
}
int main() {
    nComp = 0;
    for(int i=1; i <= N; i++) {
        if ( !idx[i] ) {
            ntime = 1;
            tarjan( i );
        }
    }
}
```

Misc.

Cermin: $\begin{bmatrix}\cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta\end{bmatrix}$ Rotasi: $\begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$

Dilatasi: $\begin{bmatrix}k & 0 \\ 0 & k\end{bmatrix}$ Gusuran/Sharing: $\begin{bmatrix}1 & k_x \\ k_y & 1\end{bmatrix}$

Regangan: $\begin{bmatrix}k_x & 0 \\ 0 & k_y\end{bmatrix}$

Gauss Elimination

```
typedef vector < vector <double> > Matrix;

const double EPS = 0.000001;

Matrix M;
bool Gaussian () {
    int N = M.size();
    for( int i=0; i<N; i++ ) {
        int idx = -1;
        for( int j = i; j < N; j++ ) {
            if (fabs (M[j][i]) > EPS) {
                idx = j;
                break;
            }
        }
        if (idx == -1){return false;}
        for( int j=i; j <= N; j++ ){swap( M[i][j],
M[idx][j] );}
        for( int j=N; j >= i; j-- ){M[i][j] /=
M[i][i];}
        for( int j=i+1; j < N; j++ ) {
```

```
            for( int k=N; k >= i; k-- ) {
                M[j][k] -= M[j][i] * M[i][k];
            }
        }
    }
    for( int i=N; i >= 0; i-- ) {
        for( int j=0; j<i; j++ ) {
            for( int k=N; k >= i; k-- ) {
                M[j][k] -= M[j][i] * M[i][k];
            }
        }
    }
    return true;
}
```

Max Bipartite Matching

```
const int MAXN = 3005;
int A[MAXN]; int na;
int B[MAXN]; int nb;

VI g[MAXN];
int v[MAXN], nomor, x1[MAXN], y1[MAXN], s[MAXN],
x2[MAXN], y2[MAXN];
int T;

bool dfs(int s) {
    if(v[s]==nomor) return false;
    v[s]=nomor;
    FOREACH(it,g[s]) {
        if( B[*it]==-1 || dfs(B[*it]) ) {
            A[s] = *it;
            B[*it] = s;
            return true;
        }
    }
    return false;
}

inline bool yes(int s) {
    nomor++;
    return dfs(s);
}

int mbm() {
    REP(i,na) A[i]=-1;
    REP(i,nb) B[i]=-1;
    int ret = 0;
    // greedy matching
    REP(i,na) {
        FOREACH(it,g[i]) if(B[*it]==-1) {
            A[i] = *it;
            B[*it] = i;
            ret++;
            break;
        }
    }
    bool ok = true;
    while(ok) {
        ok = false;
        REP(i,na) if(A[i]==-1 && yes(i)) {
            ret++;
            ok = true;
        }
    }
    return ret;
}

inline int sq(int x) { return x*x; };

inline bool can(int a,int b) {
    int d = sq(x1[a]-x2[b]) + sq(y1[a]-y2[b]);
    return sq(T*s[a]) >= d;
}
```

```
int main() {
    REP(nc,getint()) {
        T = getint();
        na = getint();
        REP(i,na) {
            x1[i]=getint();
            y1[i]=getint();
            s[i]=getint();
        }
        nb = getint();
        REP(i,nb) {
            x2[i]=getint();
            y2[i]=getint();
        }
        REP(i,na) {
            g[i].clear();
            REP(j,nb) if(can(i,j)) g[i].PB(j);
        }
        printf("Scenario #%d:\n",nc+1);
        printf("%d\n",mbm());
        puts("");
    }
}
```

Line Segment Intersection

```
struct Vertex {
    int x, y;
};
int GetArea2(Vertex& a, Vertex& b, Vertex& c) {
    return (b.x-a.x)*(c.y-a.y)-(c.x-a.x)*(b.y-a.y);
}
bool OnSegment(Vertex& a, Vertex& b, Vertex& c) {
   return(min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x)
&& min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y));
}
bool IsIntersect(Vertex& a, Vertex& b, Vertex& c,
Vertex& d) {
    int d1, d2, d3, d4;
    d1 = GetArea2(c, d, a);
    d2 = GetArea2(c, d, b);
    d3 = GetArea2(a, b, c);
    d4 = GetArea2(a, b, d);

    if(((d1 > 0 && d2 < 0) ||(d1 < 0 && d2 > 0))
&&(( d3 > 0 && d4 < 0) || d3 < 0 && d4 > 0)) {
        return true;
    }
    if(d1 == 0 && OnSegment(c, d, a)) {return true;}
    if(d2 == 0 && OnSegment(c, d, b)) {return true;}
    if(d3 == 0 && OnSegment(a, b, c)) {return true;}
    if(d4 == 0 && OnSegment(a, b, d)) {return true;}

    return false;
}
bool IsInside(Vertex& a, Vertex& lt, Vertex& rb) {
    return(min(lt.x, rb.x) <= a.x && a.x <= max(lt.x,
rb.x) && min(lt.y, rb.y) <= a.y && a.y <= max(lt.y,
rb.y));
}
// line intersection
struct Vertex {
    int x, y;
    Vertex operator - (Vertex& b) {
        Vertex result;
        result.x = x - b.x;
        result.y = y - b.y;
        return result;
    }
};
void GetIntersectionAttribute (Vertex& a, Vertex& b,
Vertex& c, Vertex& d, int& fN, int& sN, int& D) {
```

```
    fN = (d.x - c.x) * (a.y - c.y) - (d.y - c.y) *
(a.x - c.x);
    sN = (b.x - a.x) * (a.y - c.y) - (b.y - a.y) *
(a.x - c.x);
    D = (d.y - c.y) * (b.x - a.x) - (d.x - c.x) * (b.y
- a.y);
}
double GetIntersectionTime (int N, int D) {
    return (double) N / (double) D;
}
double GetIntersectionX (Vertex& a, Vertex& b, double
t) {
    return a.x + (b - a).x * t;
}
double GetIntersectionY (Vertex& a, Vertex& b, double
t) {
    return a.y + (b - a).y * t;
}
```

AVL Tree

```
// AVL Tree
struct Tree {
    int key, ubound;
    int height,size;
    Tree *left,*right;
    Tree(int key) {
        this->key = key; left = right = NULL;
        this->ubound = 0;
        height = 0; size = 1;
    }
}*root;
inline int height(Tree *p) { return p==NULL ? -1 : p-
>height; }
inline int size(Tree *p) { return p==NULL ? 0 : p-
>size; }

int upper_bound( Tree *p, int x ) {
    if ( p == NULL ) return 0;
    if ( p->key <= x ) {
        return p->ubound = size(p->left) + 1 +
upper_bound( p->right, x );
    }
    return upper_bound( p->left, x );
}

inline void leftRotate(Tree *&p) {
    Tree *c = p->left;
    p->left = c->right;
    c->right = p;
    p->height = max( height(p->left), height(p-
>right) ) + 1;
    c->height = max( height(c->left), p->height ) + 1;
    p->size = size(p->left) + size(p->right) + 1;
    c->size = size(c->left) + size(c->right) + 1;
    p = c;
}

inline void rightRotate(Tree *&p) {
    Tree *c = p->right;
    p->right = c->left;
    c->left = p;
    p->height = max( height(p->left), height(p-
>right) ) + 1;
   c->height = max( height(c->right), p->height ) + 1;
    p->size = size(p->left) + size(p->right) + 1;
    c->size = size(c->left) + size(c->right) + 1;
    p = c;
}

void insert(int x,Tree *&p) {
    if(p==NULL) p = new Tree(x);
    else if(x<=p->key) {
        insert(x,p->left);
        if(height(p->left)-height(p->right) >= 2) {
            if(x>=p->left->key) rightRotate(p->left);
            leftRotate(p);
        }
    }
    else if(x>p->key) {
        insert(x,p->right);
        if(height(p->right)-height(p->left) >= 2) {
            if(x<=p->right->key) leftRotate(p->right);
            rightRotate(p);
        }
    }
    p->height = max( height(p->left), height(p-
>right) ) + 1;
    p->size = size(p->left) + size(p->right) + 1;
    p->ubound = upper_bound( p, p->key );
}

int kth(Tree *p,int x) {
    if(size(p->left)+1==x) return p->key;
    if(x < size(p->left)+1) return kth(p->left,x);
    return kth(p->right,x-size(p->left)-1);
}

void printAll( Tree *p ) {
    if ( p == NULL ) return;
    printAll( p->left );
    printf (" %d ", p->key );
    printAll( p->right );
}

inline Tree * findMin(Tree *p) {
    while(p->left!=0) p=p->left;
    return p;
}

void removeAll( Tree *p ) {
    if ( p==NULL ) return;
    removeAll( p->left );
    removeAll( p->right );
    delete p;
}

void remove(int x,Tree *&p) {
    if(p==0) return;
    if(x<p->key) remove(x,p->left);
    else if(x>p->key) remove(x,p->right);
    else if(p->left!=0 && p->right!=0) {
        p->key = findMin(p->right)->key;
        remove(p->key,p->right);
    }
    else {
        Tree *temp = p;
        p = (p->left!=0) ? p->left:p->right;
        delete temp;
    }
    if(p!=0) {
        p->size = size(p->left) + size(p->right) + 1;
        p->height = max(height(p->left),height(p-
>right))+1;
    }
}
```

Kadane

```
// Kadane 2D
    int N, t = 0, a[100][100], pr[100];
    int S = 1<<31, s = 0, k, l, x1 = 0,x2 = 0,y1 =
0,y2 = 0,j;
    for( int z = 0; z < N; z++) {
        for(int i = 0; i < N; i++) { pr[i] = 0; }
        for(int x = z; x < N; x++) {
            t = j = k = l = 0, s = 1<<31;
            for(int i = 0; i < N; i++) {
```

```
                pr[i] = pr[i] + a[x][i];
                t = t + pr[i];
                if( t > s) {
                    s = t, k = i,l = j;
                }
                if( t < 0 ) {
                    t = 0, j = i + 1;
                }
            }
            if( s > S) {
                S = s;
                x1 = x;
                y1 = k;
                x2 = z;
                y2 = l;
            }
        }
    }
    cout << x1 << " " << y1 << " " << x2 << " "  << y2
<< endl;
    cout << S;
}
// 1D
    (k; l) := (0; 0); s := -inf; t := 0; j := 1;
    for i:=1 to n do begin
        t := t + a[i];
        if t > s then begin (k; l) := (j; i); s := t
end;
        if t < 0 then begin t := 0; j := i + 1 end
    end
```

Soal Timo 2010

```
const LL mod = 1000000007;
LL  nck[100][100];
LL  memo[10][100];
int arr[10];
LL f(int x, int m) {
    if ( m ==  0 ) return 1;
    if ( x == 10 ) return 0;
    if ( memo[x][m] != -1 ) return memo[x][m];
    LL &ret = memo[x][m] = 0;
    FOR(i,0,min(m,arr[x])) ret = (ret + (f(x+1,m-i) *
nck[m][i]) % mod) % mod;
    return ret;
}
int main(){
    nck[0][0] = 1;
    FOR(i,1,99) {
        nck[i][0] = nck[i][i] = 1;
        FOR(j,1,i-1) nck[i][j] = (nck[i-1][j] + nck[i-
1][j-1]) % mod;
    }
    int T;
    scanf( "%d", &T );
    while ( T-- ) {
        FOR(i,1,9) scanf( "%d", &arr[i] );
        int total = accumulate(arr+1,arr+10,0);
        LL ans = 0;
        FOR(k,1,9) if ( arr[k] != 0 ) {
            arr[k]--;
            memset(memo,-1,sizeof(memo));
            LL p = k;
            FOR(len,1,total) {
                ans = (ans + (f(1,len-1) * p) % mod) %
mod;
                p   = (p * 10 + k) % mod;
            }
            arr[k]++;
        }
        printf( "%I64d\n", ans );
    }
}
```

```
// Abridged problem statement: Given the number of
available digits of digit 1 to 9, sum all possible
numbers that can be generated by those available
digits.
```

Meet in the middle

```
    long sumsLowerBound(long[] A, long B) {
        int n = A.length;
        int sizeL = 1 << (n / 2),sizeR=1<<(n - n / 2);
      long[] SumsL=new long[sizeL],SumsR=new long[sizeR];
        for (int i = 0; i < sizeL; ++i){
            for (int j = 0; j < n / 2; ++j){
                if ((i & (1 << j)) > 0){
                    SumsL[i] += A[j];}}}
        for (int i = 0; i < sizeR; ++i){
            for (int j = 0; j < n - n / 2; ++j){
                if ((i & (1 << j)) > 0){
                    SumsR[i] += A[j + n / 2];}}}
        Arrays.sort(SumsL);
        Arrays.sort(SumsR);
        int PL = 0, PR = sizeR - 1;
        long cur = Long.MIN_VALUE;
        while (PL < sizeL && PR >= 0) {
            if (SumsL[PL] + SumsR[PR] <= B) {
                cur = Math.max(cur, SumsL[PL] +
SumsR[PR]);
                ++PL;
            } else{
                --PR;}
        }
        return cur;}
```